



HOME CREDIT



Programme

Presentation desq données

Exploration nettoyage des
données

preparation des données

Feature engineering

Models

comparaison

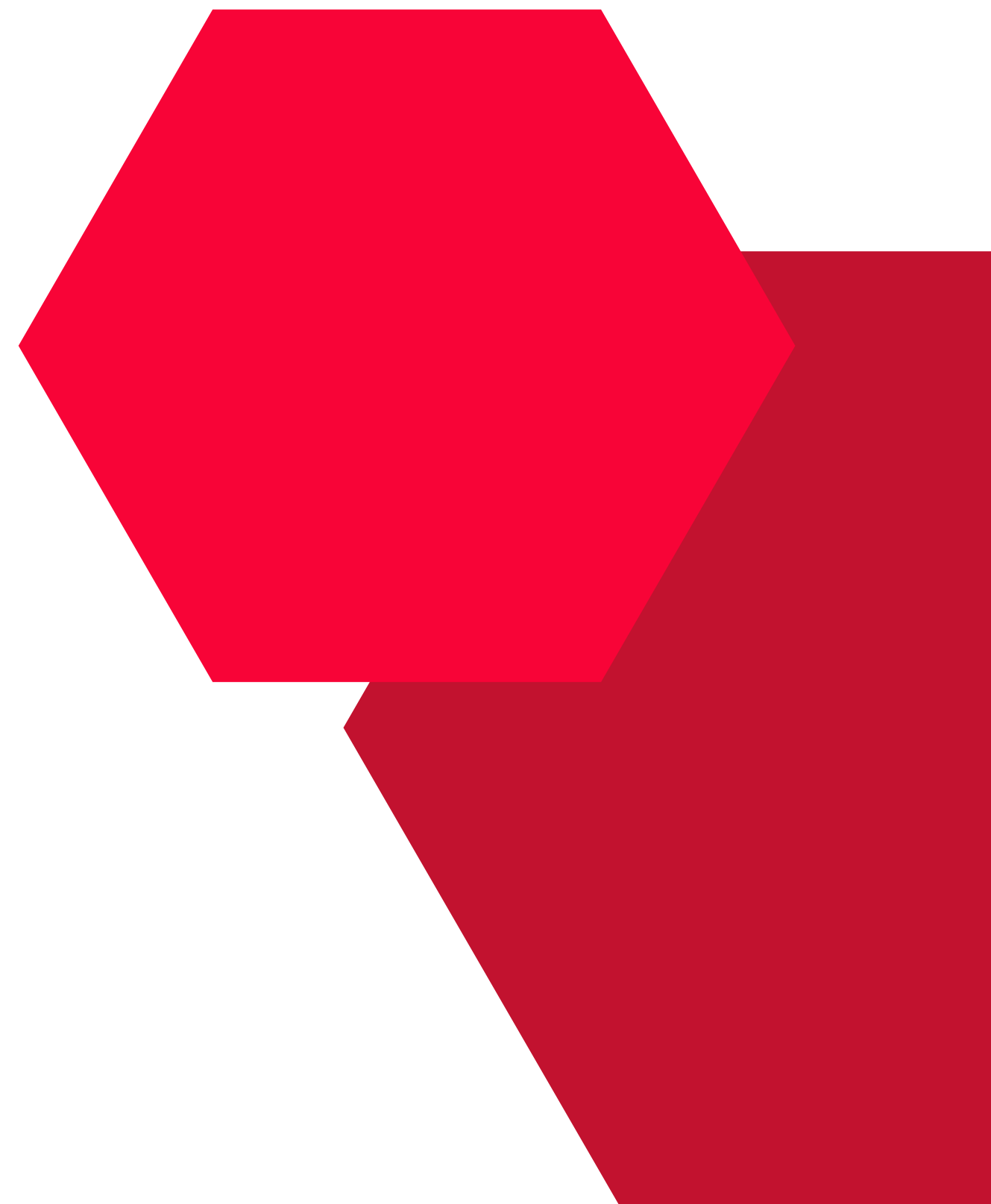
visualisation

**HOME
CREDIT
BANK**



Prêt à dépenser

Pour accorder un crédit à la consommation, l'entreprise souhaite mettre en œuvre un outil de “scoring crédit” qui calcule la probabilité qu'un client le rembourse ou non, puis classifie la demande : crédit accordé ou refusé.



STEP TO BUILD MACHINE LEARNING MODEL



Acquisition
des
données

Preparation
&
Nettoyage

Analyse
&
Exploration

Features
engineering

Choix du model

Visualisation
des
Resultas



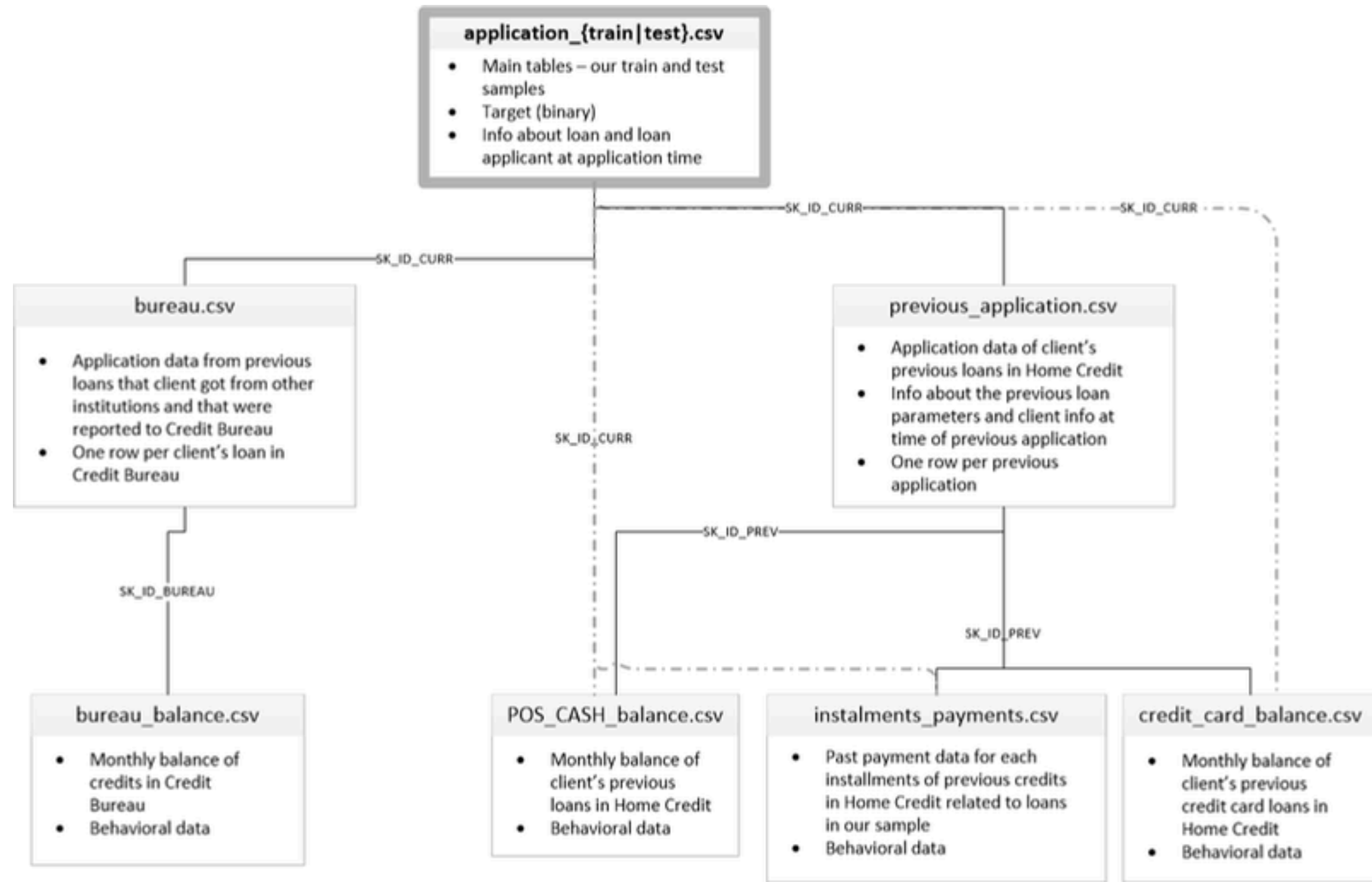
EXPLORATION DES DONNEES

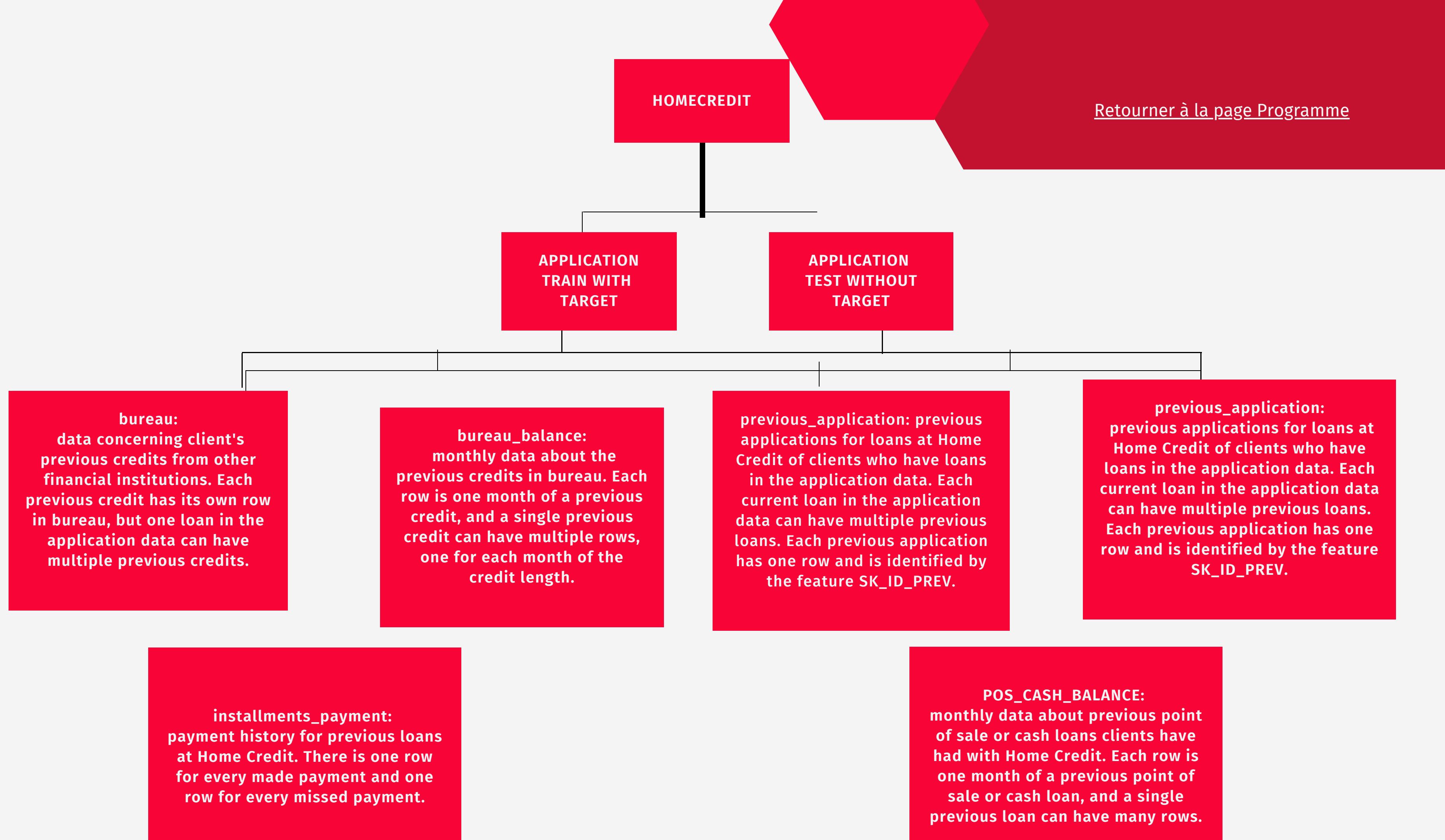
Soutenez votre argument avec une brève explication.



[Retourner à la page Programme](#)

DATASETS





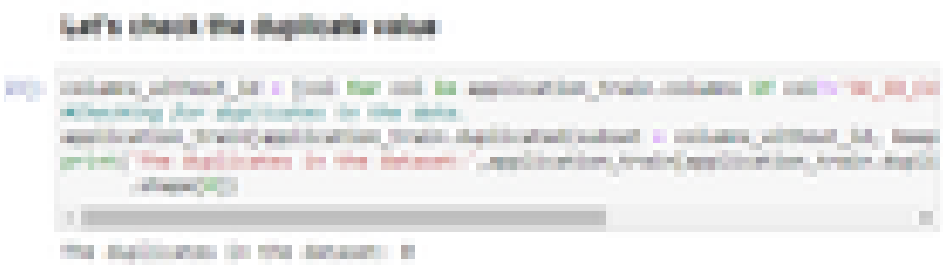
NETTOYAGE

- Gestion des doublons.
- Gestion des NAN.
- .Gestion des données Aberantes
- Gestion des données les moins renseignées



Nettoyage des données

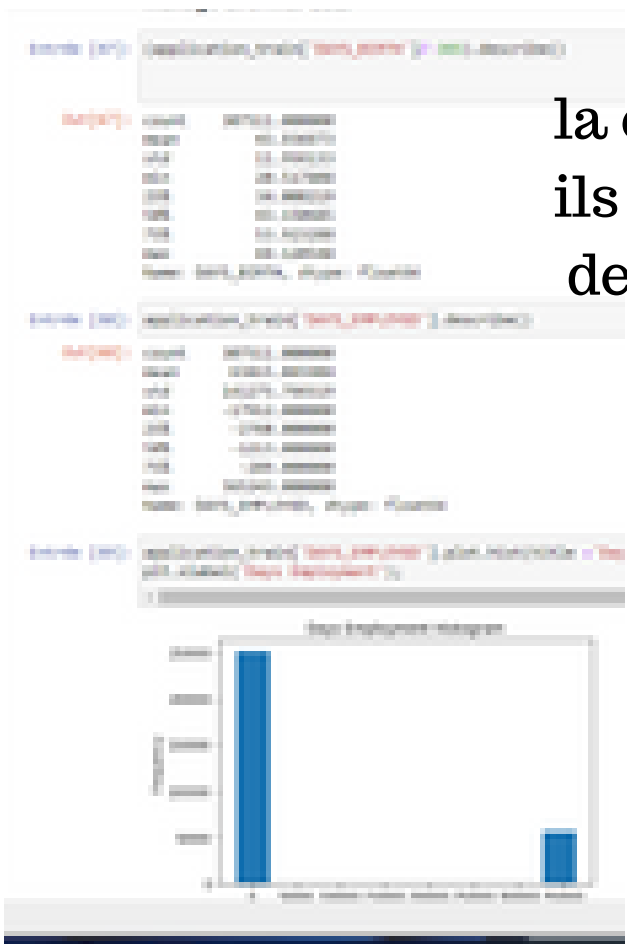
Verification de doublons



Gestion des NAN imputation par la moyenne
Suppression des variables les moins renseignées

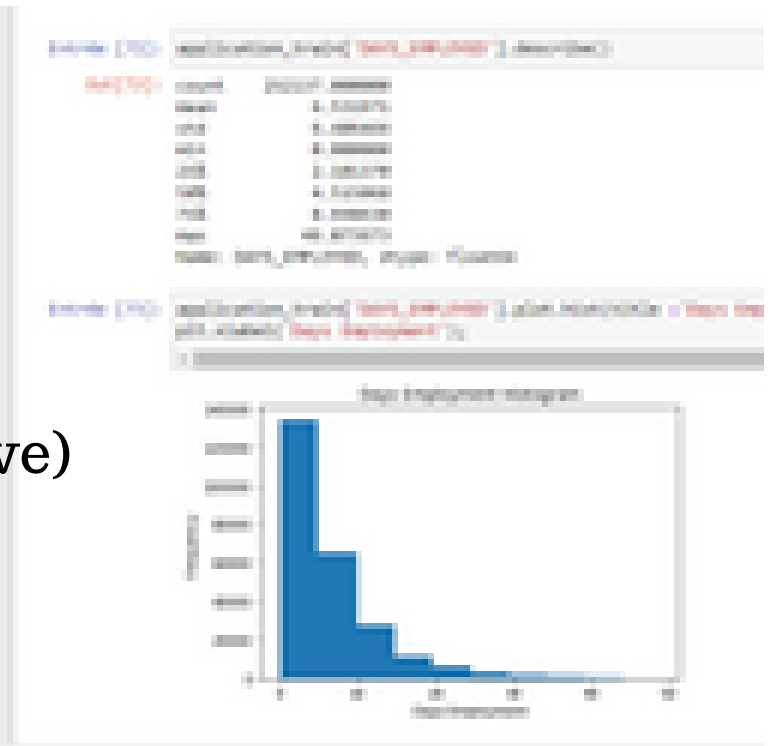


Gestion des données aberrantes

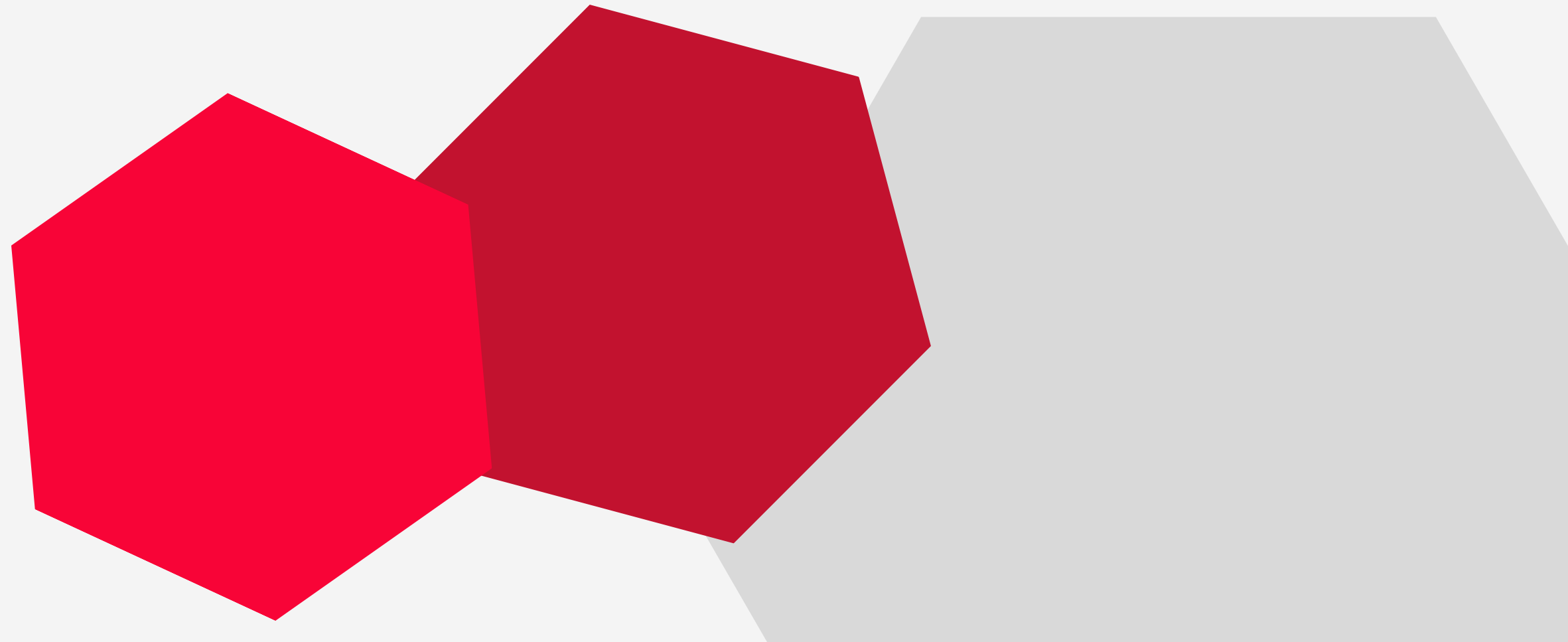


la colonne DAYS_BIRTH sont négatifs car ils sont enregistrés par rapport à la demande de prêt en cours

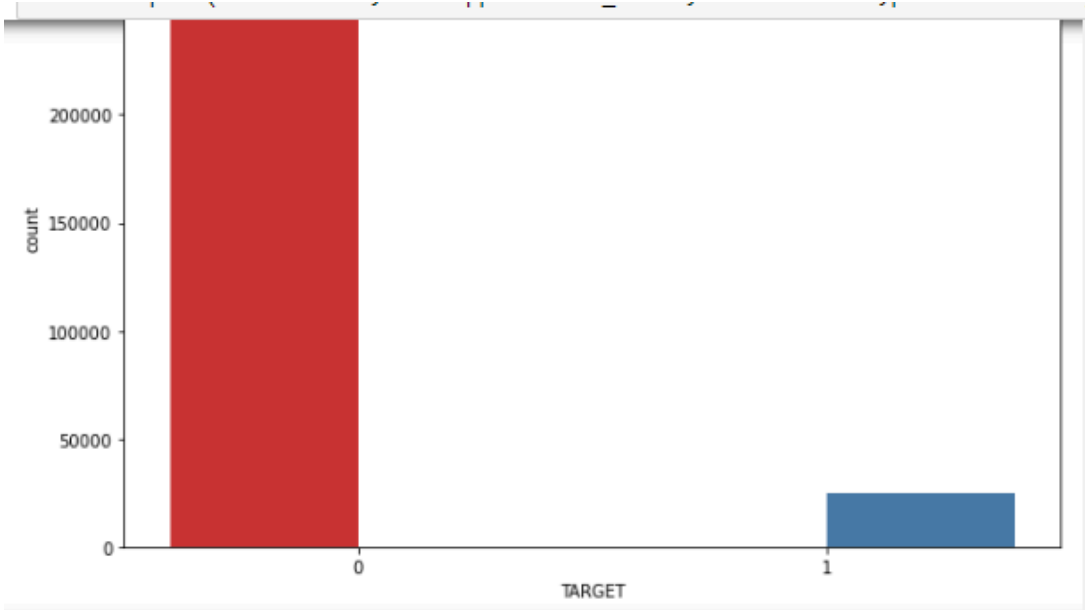
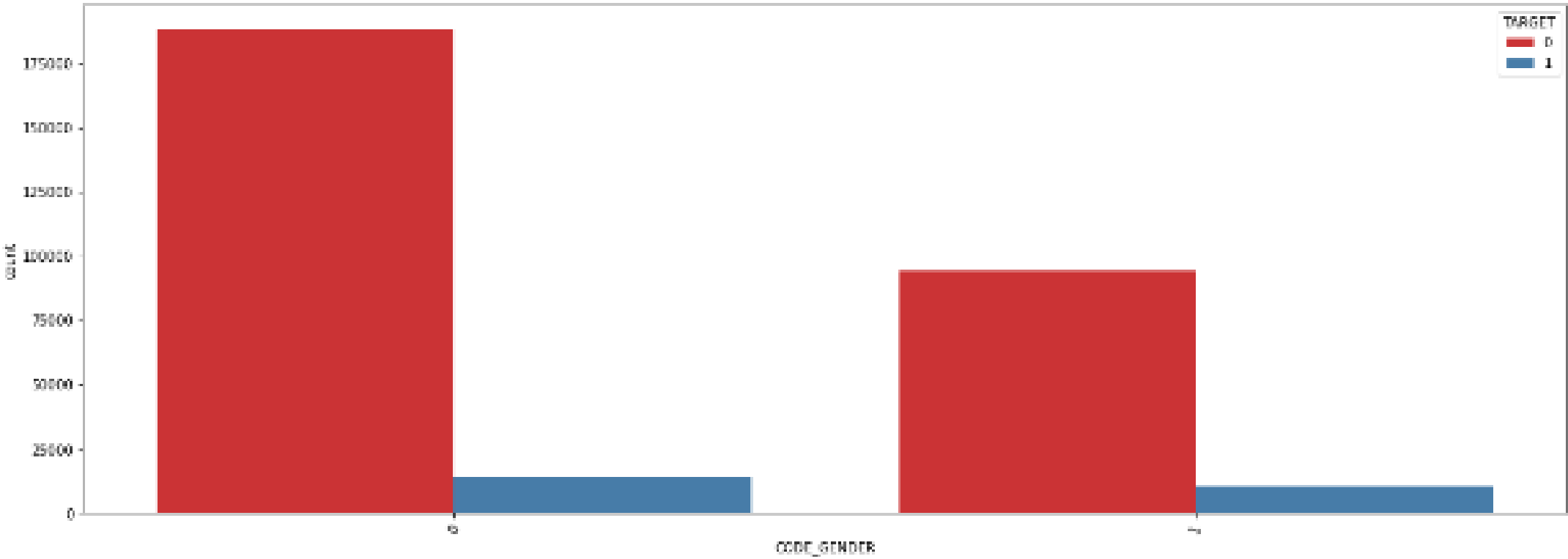
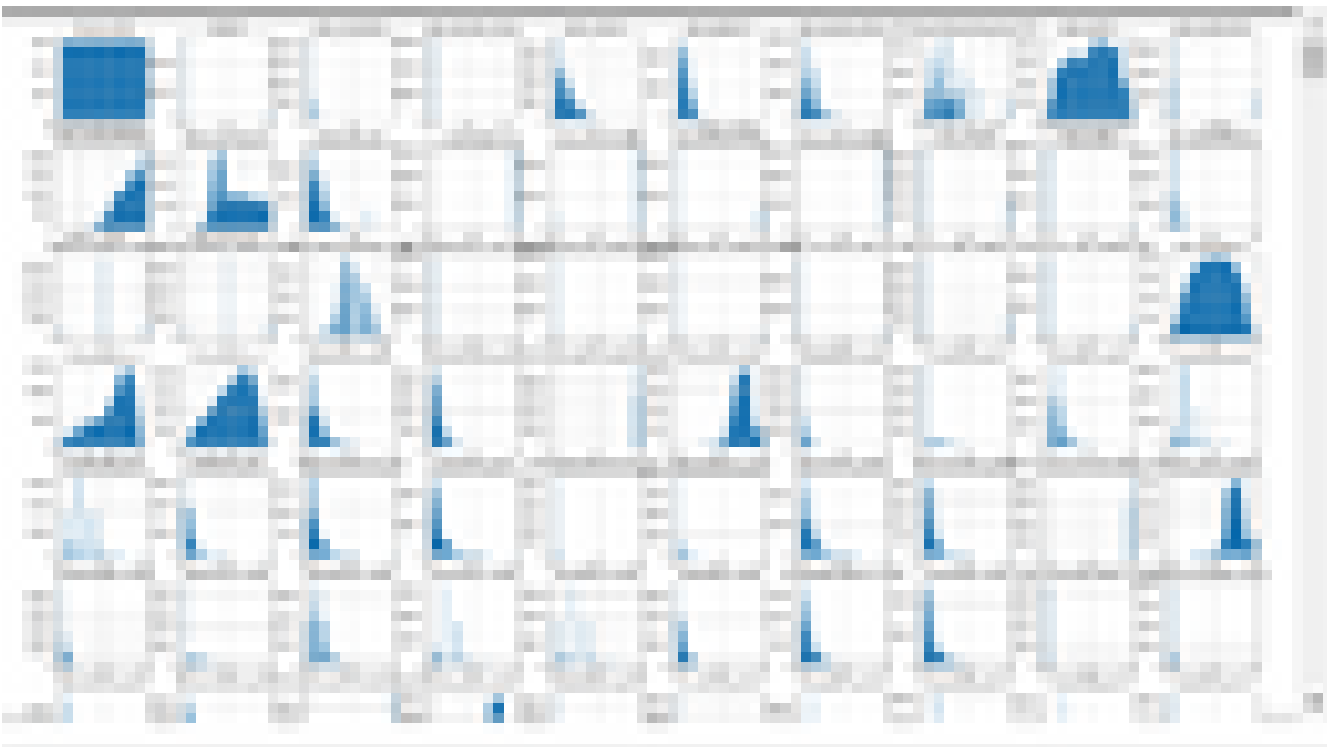
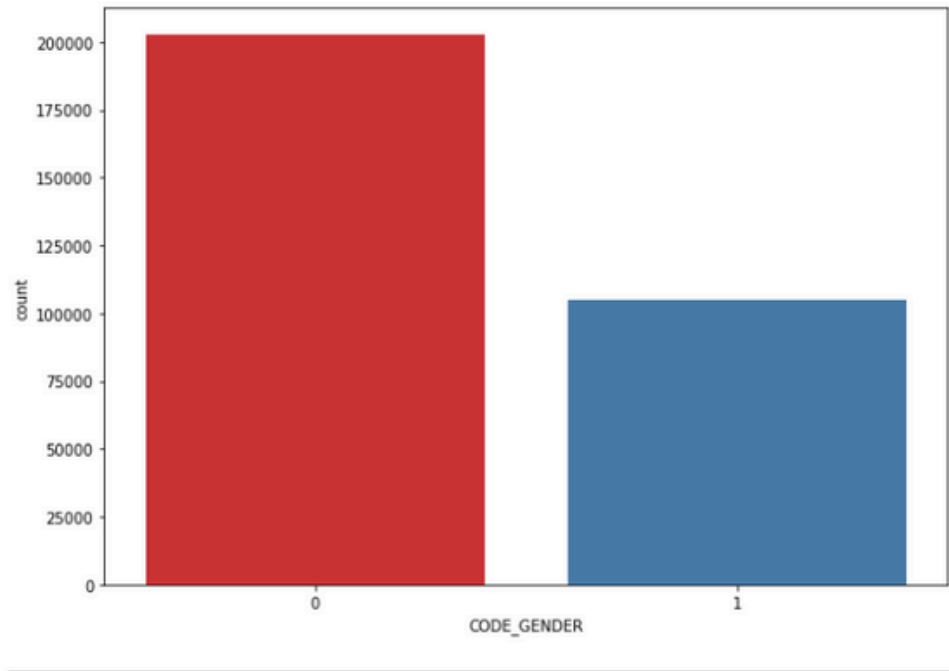
La valeur maximale (en plus d'être positive) est d'environ 1000 ans !



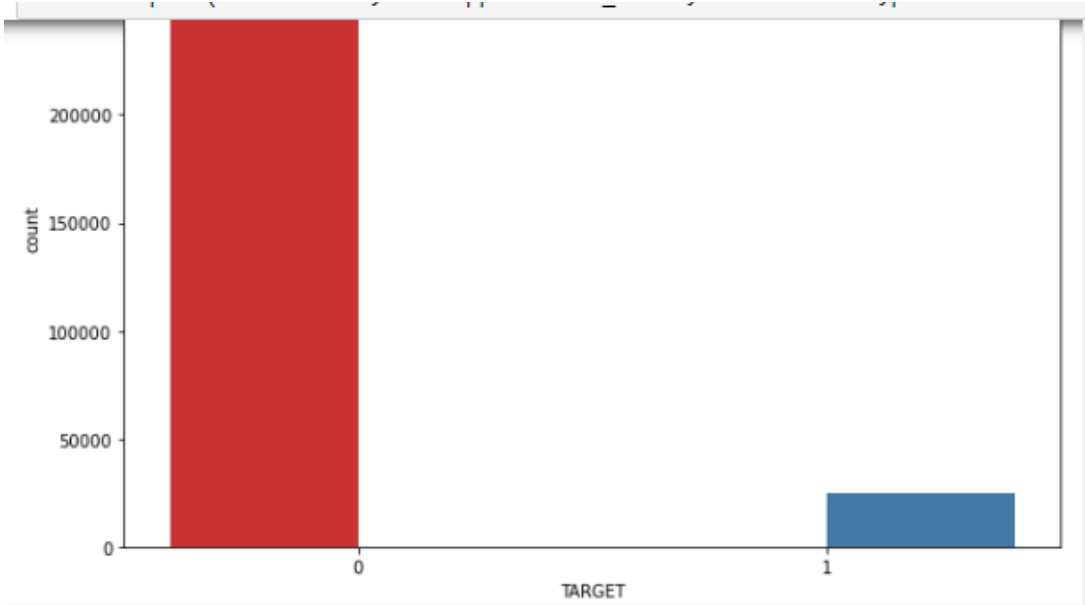
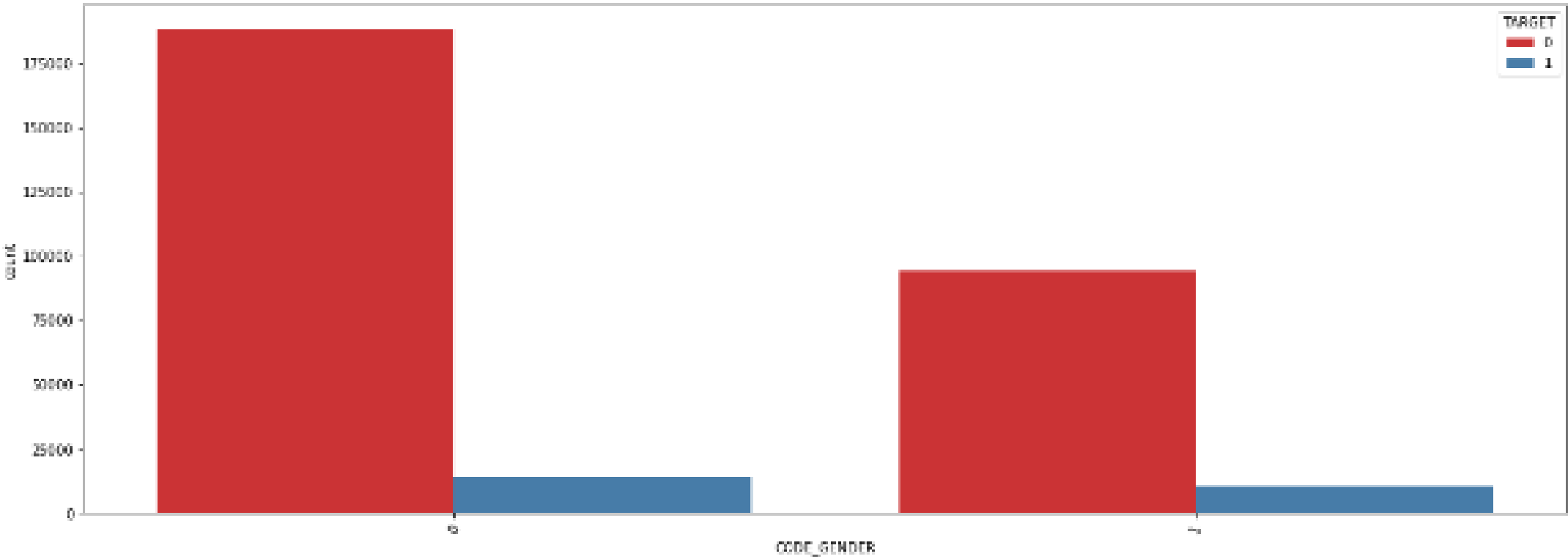
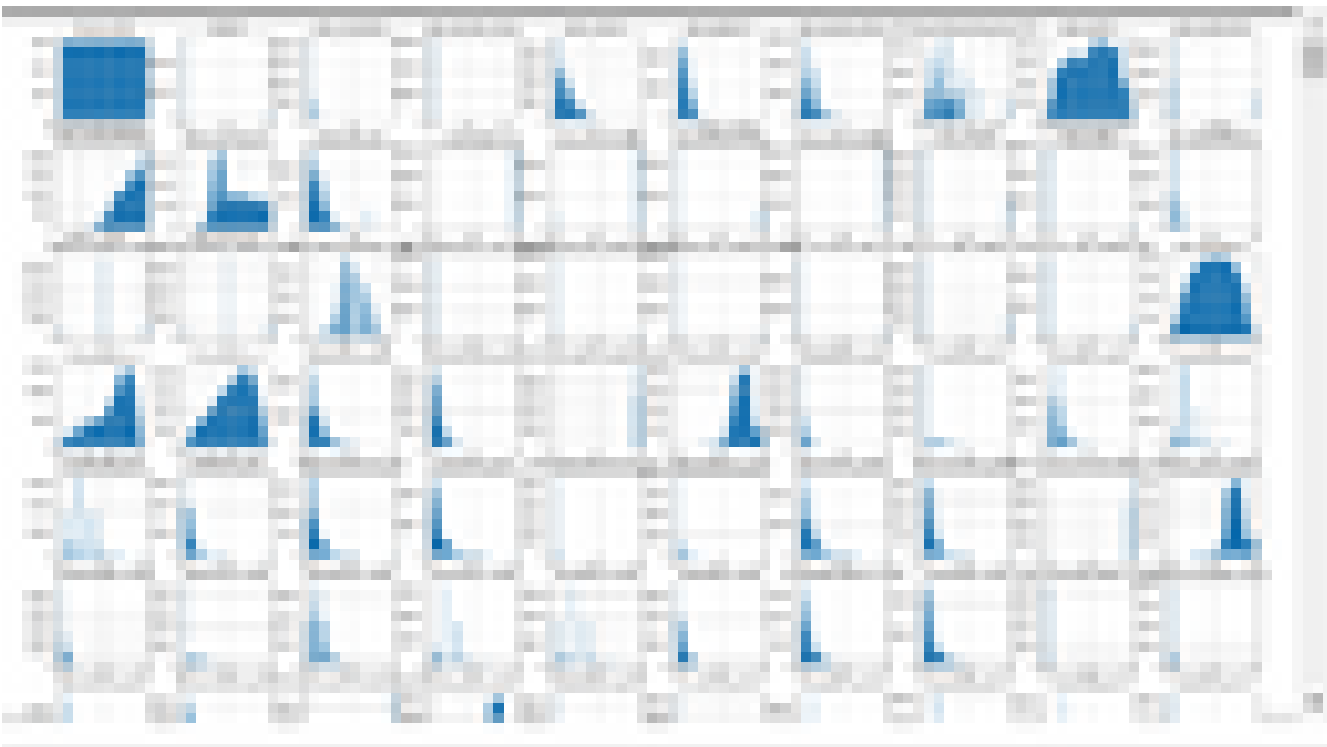
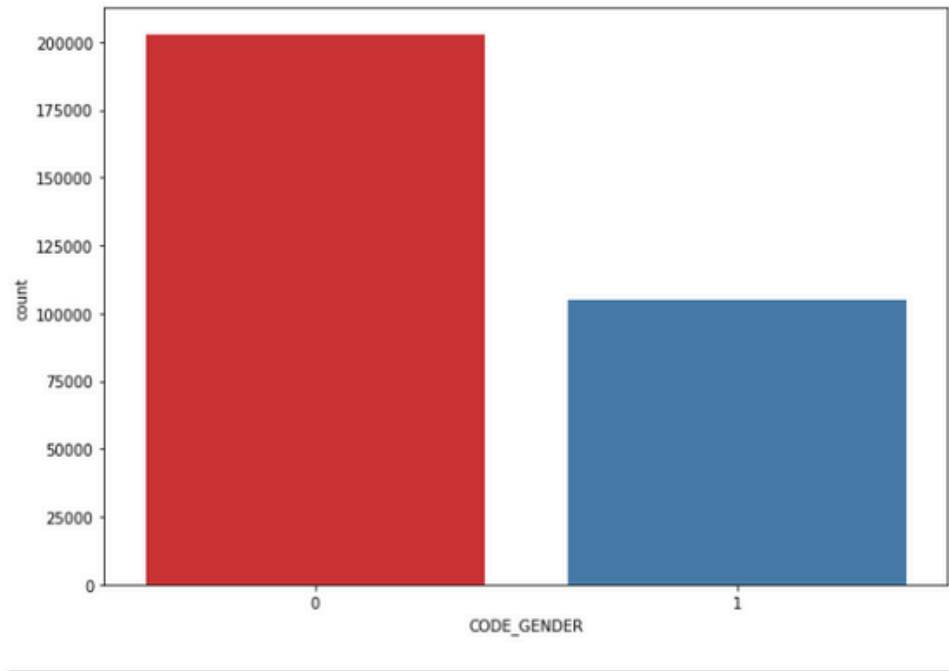
ANALYSE EDA

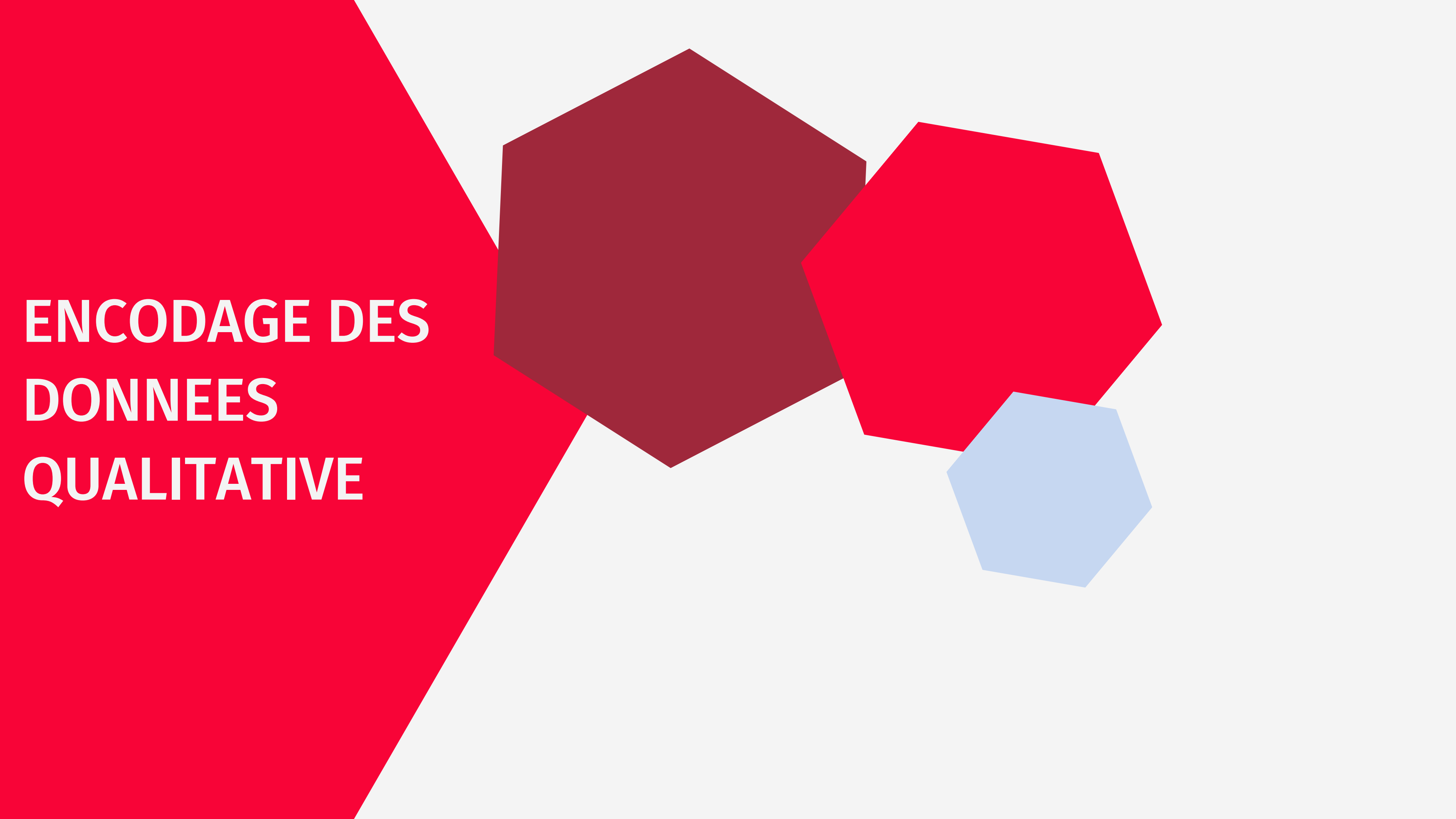


EDA ANALYSE



EDA ANALYSE





**ENCODAGE DES
DONNEES
QUALITATIVE**

Encodage Des donnes

Label encoding les variables avec deux valeur

affectez chaque catégorie unique dans une variable catégorielle avec un nombre entier. Aucune nouvelle colonne n'est créée. Un exemple est montré ci-dessous

```
: # one-hot encoding of categorical variables > 2
application_train = pd.get_dummies(application_train)
application_test = pd.get_dummies(application_test)

print('Training Features shape: ', application_train.shape)
print('Testing Features shape: ', application_test.shape)
```

```
Training Features shape: (307511, 241)
Testing Features shape: (48744, 238)
```

One-hot encoding pour les variables avec plus de deux valeurs

créez une nouvelle colonne pour chaque catégorie unique dans une variable catégorielle. Chaque observation reçoit un 1 dans la colonne de sa catégorie correspondante et un 0 dans toutes les autres nouvelles colonnes.

```
: from sklearn.preprocessing import LabelEncoder
# Create a Label encoder object
le = LabelEncoder()
le_count = 0

# Iterate through the columns
for col in application_train:
    if application_train[col].dtype == 'object':
        # If 2 or fewer unique categories
        if len(list(application_train[col].unique())) <= 2:
            # Train on the training data
            le.fit(application_train[col])
            # Transform both training and testing data
            application_train[col] = le.transform(application_train[col])
            application_test[col] = le.transform(application_test[col])

            # Keep track of how many columns were Label encoded
            le_count += 1

print('%d columns were label encoded.' % le_count)
```

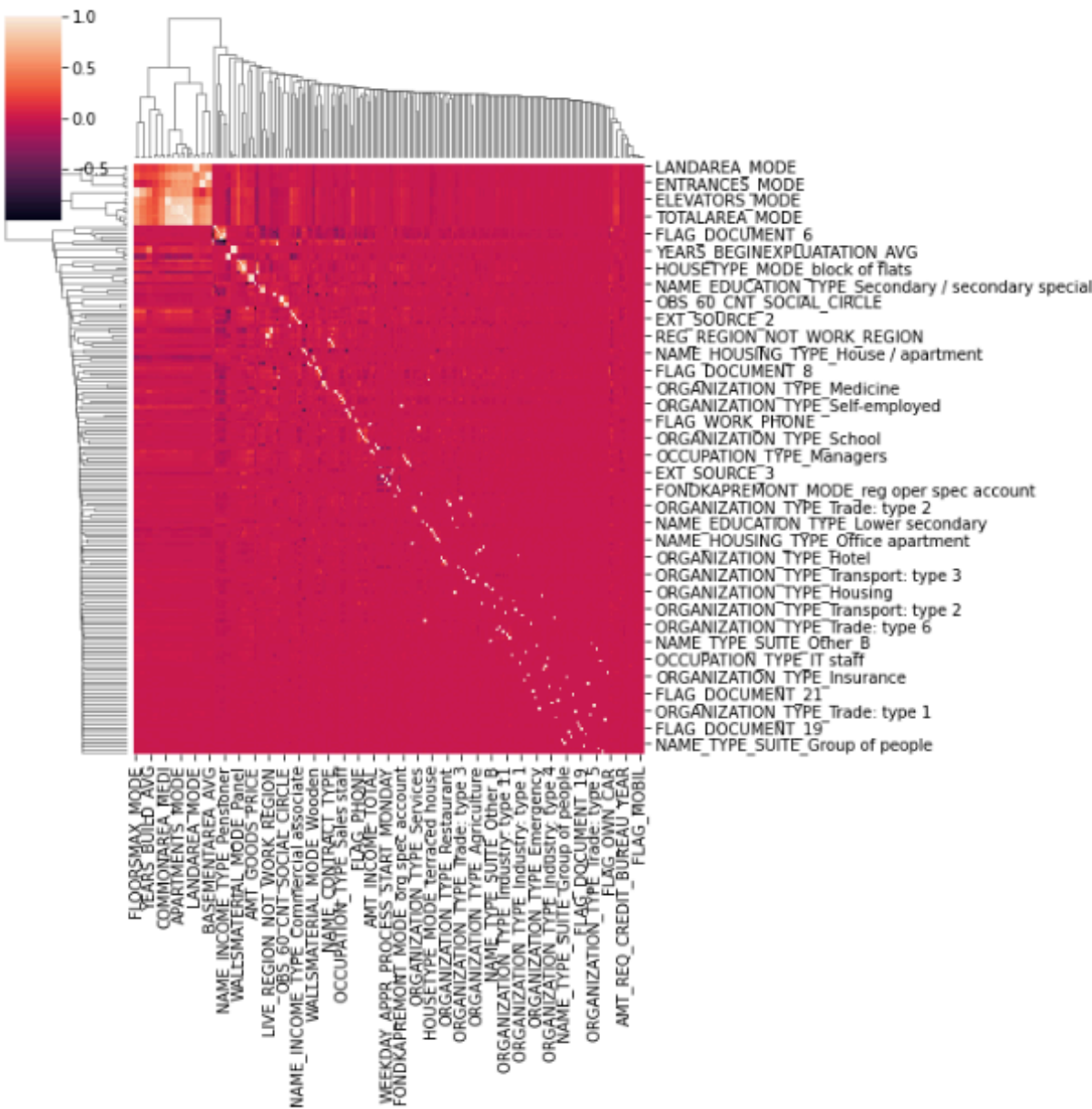
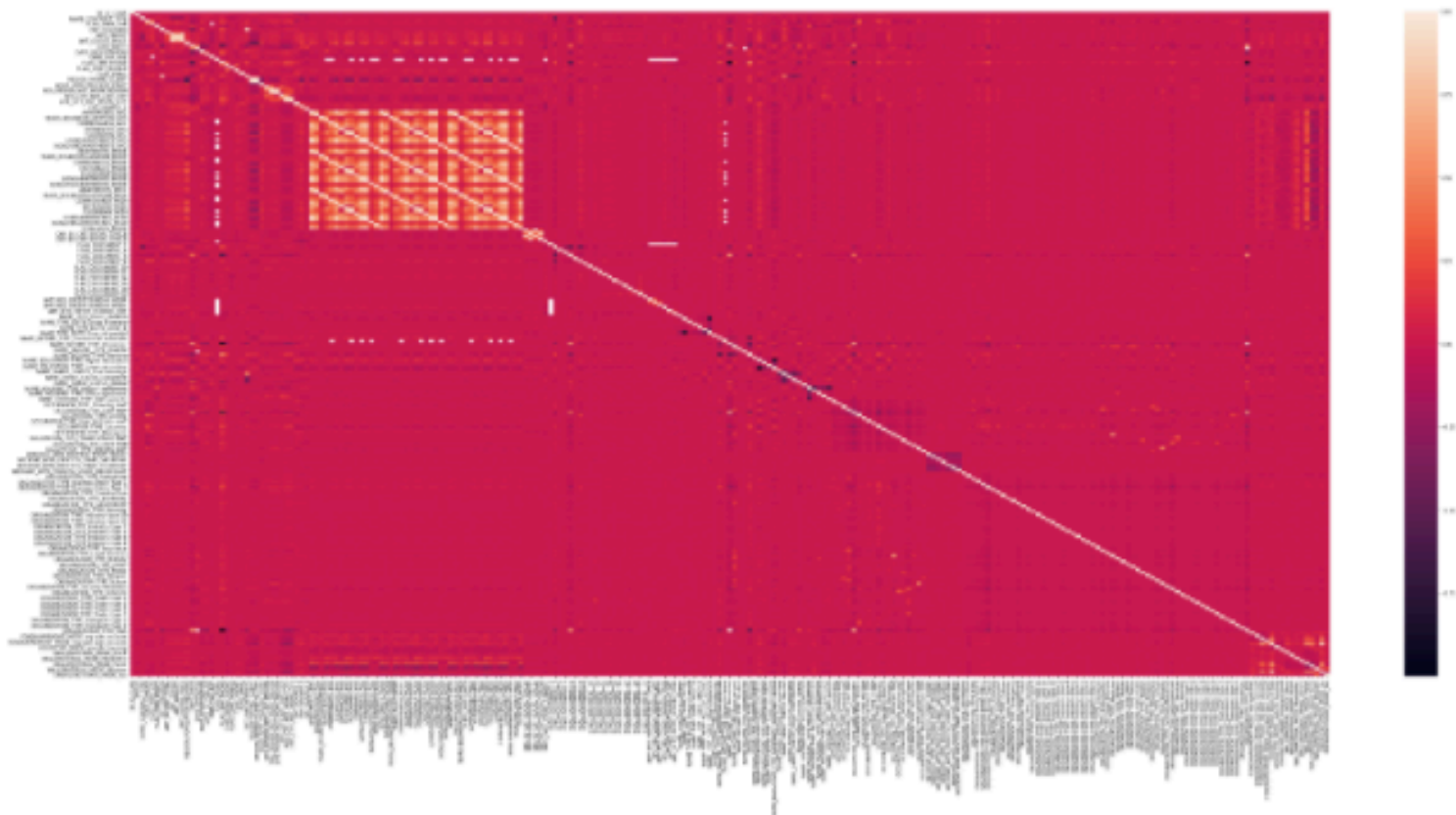
```
4 columns were label encoded.
```



ANALYSE DES CORRELATIONS

Analyse des correlations

L'analyse des correlations permet de reduire la taille du dataset selon l'utilité de la donnée notement correlation avec la Variable TARGET et les correlations par paire pour supprimer l'une des deux



Analyse des correlations

Suppression des valeur tres ou peu correlée a TARGET

```
|: # Let's find variables that are highly de-correlated from TARGET
corr_target_min_threshold = 0.01
high_target_decorrelated = pd.DataFrame(
    columns=["correlation with TARGET"]
)
for col in correlation.columns:
    if col != "TARGET" and (
        pd.isnull(correlation[col]["TARGET"])
        or abs(correlation[col]["TARGET"]) < corr_target_min_threshold
    ):
        high_target_decorrelated.loc[col] = {
            "correlation with TARGET": correlation[col]["TARGET"],
        }

high_target_decorrelated.sort_values(by=["correlation with TARGET"])
```

correlation with TARGET	
YEARS_BEGINEXPLUATATION_MEDI	-0.009993
ORGANIZATION_TYPE_Police	-0.009886
YEARS_BEGINEXPLUATATION_AVG	-0.009728
ORGANIZATION_TYPE_Bank	-0.009609
ORGANIZATION_TYPE_Security Ministries	-0.009470
...	...
ORGANIZATION_TYPE_Trade: type 3	0.008911
CNT_FAM_MEMBERS	0.009308
OBS_60_CNT_SOCIAL_CIRCLE	0.009513
OBS_30_CNT_SOCIAL_CIRCLE	0.009628
ORGANIZATION_TYPE_Industry: type 3	0.009691

```
pair_corr_max_threshold = 0.9
highly_correlated = pd.DataFrame(columns=["pair", "correlation"])
for i in range(len(correlation.columns)):
    for j in range(i + 1, len(correlation.columns)):
        if correlation.iloc[i, j] > pair_corr_max_threshold:
            # variables are highly correlated
            if correlation.iloc[0, i] > correlation.iloc[0, j]:
                # first variable is more correlated with target
                # => we want to keep it
                keep_index = i
                drop_index = j
            else:
                keep_index = j
                drop_index = i

        highly_correlated.loc[correlation.columns[drop_index]] = {
            "pair": correlation.columns[keep_index],
            "correlation": correlation.iloc[i, j],
        }

highly_correlated.sort_values(by="correlation", ascending=False)
```

Out[28]:

		pair	correlation
ORGANIZATION_TYPE_XNA	NAME_INCOME_TYPE_Pensioner		0.999648
YEARS_BUILD_MEDI	YEARS_BUILD_AVG		0.998495
OBS_60_CNT_SOCIAL_CIRCLE	OBS_30_CNT_SOCIAL_CIRCLE		0.998378
FLOORSMIN_MEDI	FLOORSMIN_AVG		0.997241
FLOORSMAX_MEDI	FLOORSMAX_AVG		0.997034
ENTRANCES_AVG	ENTRANCES_MEDI		0.996886
ELEVATORS_AVG	ELEVATORS_MEDI		0.996099
COMMONAREA_AVG	COMMONAREA_MEDI		0.995978
BASEMENTAREA_AVG	BASEMENTAREA_MEDI		0.994317
LIVINGAPARTMENTS_AVG	LIVINGAPARTMENTS_MEDI		0.993825
LANDAREA_AVG	LANDAREA_MEDI		0.991610
NONLIVINGAREA_MEDI	NONLIVINGAREA_AVG		0.990444
YEARS_BUILD_MODE	YEARS_BUILD_MEDI		0.989463
FLOORSMIN_MODE	FLOORSMIN_MEDI		0.988406
FLOORSMAX_MODE	FLOORSMAX_MEDI		0.988237

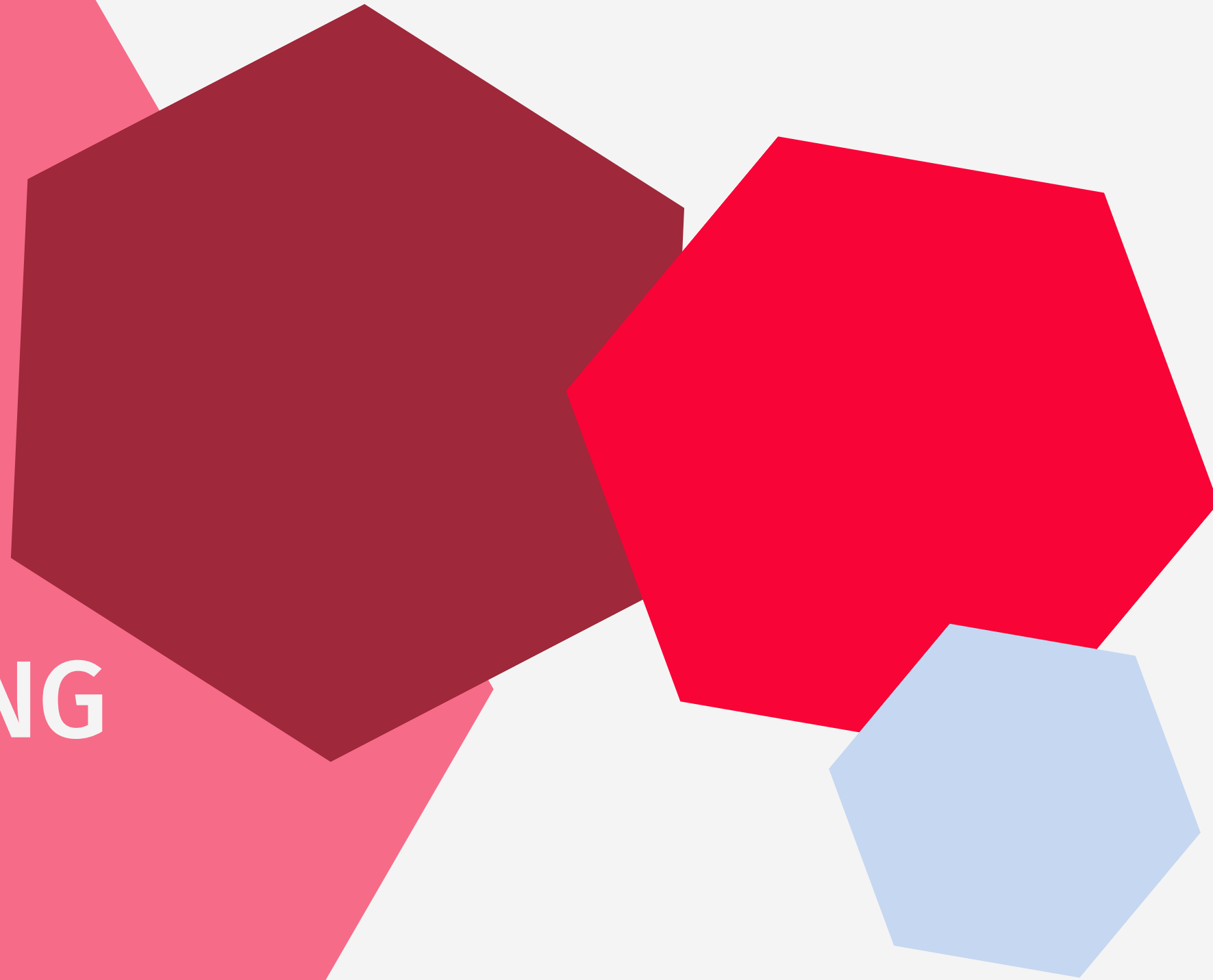
```
: # Drop irrelevant columns
app_train_business.drop(
    columns=high_target_decorrelated.index,
    inplace=True,
    errors="ignore",
)
app_train_business.drop(
    columns=highly_correlated.index,
    inplace=True,
    errors="ignore",
)

app_train_business.describe(include="all")
```

	TARGET	NAME_CONTRACT_TYPE	CODE_GENDER	FLAG_OWN_CAR	CNT_CHILDREN
count	307511.000000	307511.000000	307511.000000	307511.000000	307511.000000
mean	0.080729	0.095213	0.341656	0.340108	0.417052
std	0.272419	0.293509	0.474266	0.473746	0.722121
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000	0.000000
50%	0.000000	0.000000	0.000000	0.000000	0.000000
75%	0.000000	0.000000	1.000000	1.000000	1.000000
max	1.000000	1.000000	1.000000	1.000000	19.000000

Annalyse par paire et supprimer l'une des variable tres correlée

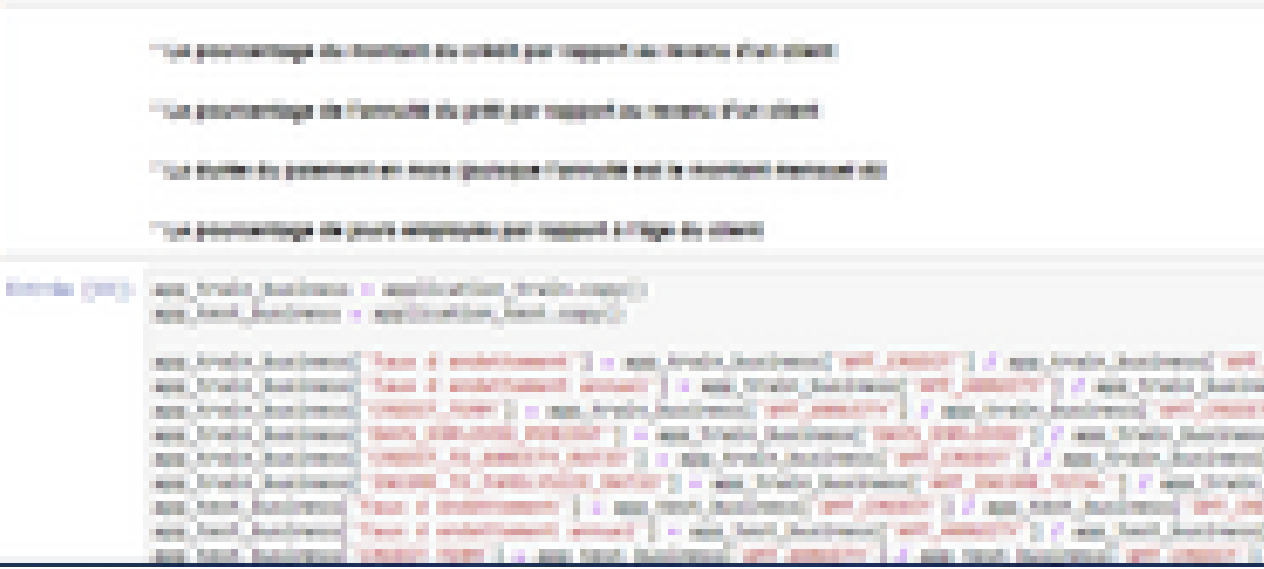
FEATURE ENGINEERING



FEATURE ENGINEERING

Pour donner du sens à notre Modèle, enrichissons-le avec des données métiers utiles à la prise de décision

- * Le taux d'endettement ne doit pas dépasser selon la banque 33% ou maximum 35%
- * Si possible calculez le reste à vivre (nous n'avons que le total des crédits, mais pas le total des ressources, ni le total des dépenses nous pouvons utiliser le total des crédits) si un demandeur a trop de prêts il pourrait avoir des difficultés financières.
- * Vérifier si sur tous les crédits passés il y a eu des incidents de paiement ou non
- * Et le montant que l'emprunteur doit payer à la banque au moment du défaut
- * Total des défauts de paiement
- * Nombre de fois que l'emprunteur a été en retard de 30 à 59 jours, mais pas pire au cours des 2 dernières années.



```
def feature_engineering(features):
    # Calcul du taux d'endettement
    features['taux_endettement'] = (features['montant_credits'] / features['revenu_mensuel']) * 100

    # Calcul du reste à vivre
    features['reste_a_vivre'] = features['revenu_mensuel'] - features['montant_credits']

    # Calcul du nombre de défauts de paiement
    features['nombre_defauts'] = features['historique_paiements'].apply(lambda x: x['defauts'].count())

    # Calcul du nombre de retards de 30 à 59 jours
    features['nombre_retards'] = features['historique_paiements'].apply(lambda x: x['retards_30_59_jours'].count())

    return features
```

FEATURE ENGINEERING

Pour donner du sens à notre Modèle, enrichissons-le avec des données métiers utiles à la prise de décision

- *Le taux d'endettement ne doit pas dépasser

selon la banque 33% ou maximum 35%

- * Si possible calculez le reste à vivre (nous n'avons que le total des crédits, mais pas le total des ressources

, ni le total des dépenses nous pouvons utiliser le total des crédits)

si un demandeur a trop de prêts

il pourrait avoir des difficultés financières.

- * Vérifier si sur tous les crédits passés il y a eu des incidents de paiement ou non

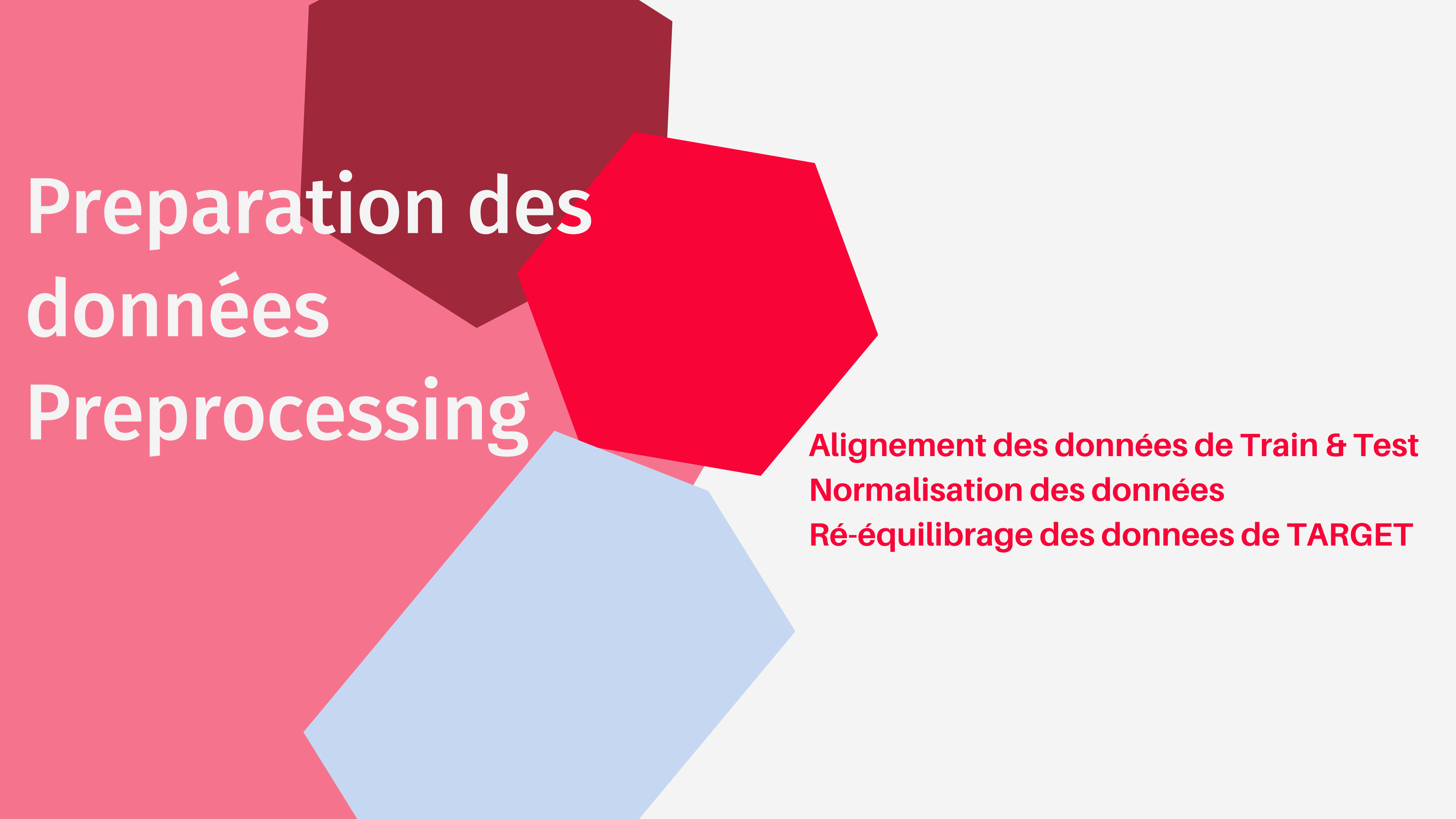
- * Et le montant que l'emprunteur doit payer à la banque au moment du défaut

- * Total des défauts de paiement

- *Nombre de fois que l'emprunteur a été

en retard de 30 à 59 jours, mais pas pire au cours des 2 dernières années.

```
def feature_engineering(features):  
    # Calcul du taux d'endettement  
    features['taux_endettement'] = features['montant_credits'] / features['revenu_mensuel']  
    # Calcul du reste à vivre (si possible)  
    features['reste_a_vivre'] = features['revenu_mensuel'] - features['montant_credits']  
    # Calcul du total des crédits  
    features['total_credits'] = features['montant_credits']  
    # Calcul du total des dépenses (si possible)  
    features['total_depenses'] = features['montant_credits']  
    # Calcul du total des défauts de paiement  
    features['total_defauts'] = features['montant_credits']  
    # Calcul du nombre de fois que l'emprunteur a été en retard  
    features['nombre_retards'] = features['montant_credits']
```

Préparation des données Preprocessing

Alignement des données de Train & Test
Normalisation des données
Ré-équilibrage des données de TARGET

Alignement des données de Train & Test

Ne conservez que les colonnes présentes dans les deux dataframes

Aligner les ensembles de données consiste à garder que les colonnes présentes dans les deux jeu de données train and test

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Load training and testing data
train_data = pd.read_csv('train_data.csv')
test_data = pd.read_csv('test_data.csv')

# Identify common columns between train and test data
common_columns = train_data.columns.intersection(test_data.columns)

# Filter the data to keep only common columns
train_data = train_data[common_columns]
test_data = test_data[common_columns]

# Standardize the data
scaler = StandardScaler()
train_data = scaler.fit_transform(train_data)
test_data = scaler.transform(test_data)
```

Normalisation des données

La normalisation est une opération de pré-traitement de données qui a pour but de mettre sur une même échelle toutes les variables quantitatives.

Si deux variables ne sont pas sur une même échelle de grandeur, la comparaison n'aura pas véritablement de sens. Une mise à échelle est nécessaire pour mettre nos variables sur la même échelle, c'est ce qui facilitera la comparaison et lui donnera du sens et ne faussera pas l'apprentissage

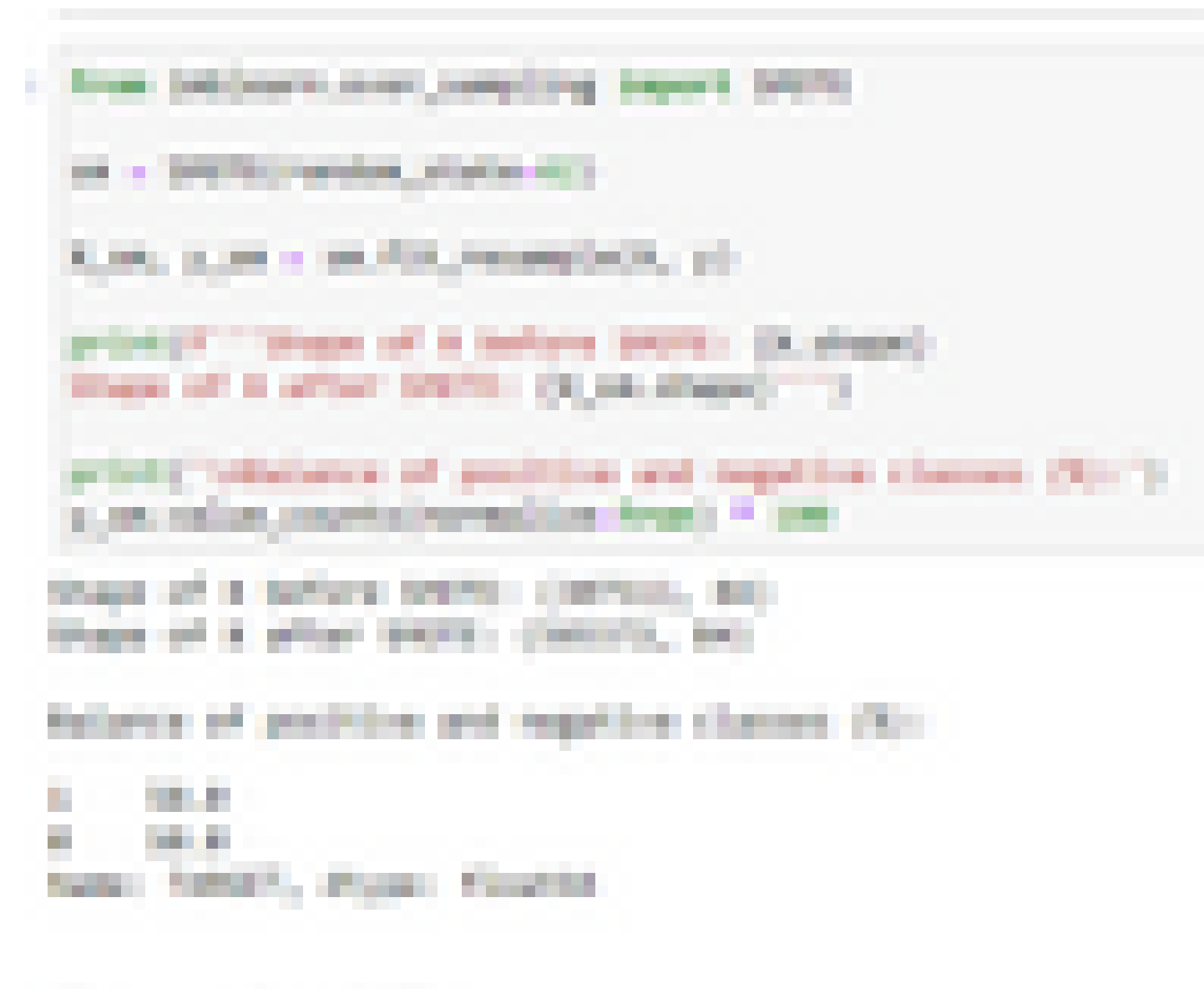


Ré-équilibrage des données

TARGET sont déséquilibrées, Nous allons donc ré-équilibrer ces classes dans nos jeux d'apprentissage afin de palier à ce problème.

SMOTE: Synthetic Minority Over-sampling Technique)

L'algorithme permet de générer plus de données de la classe sous-représentée puis éliminer certains individus de la classe sur-représentées (ENN : Edited Nearest Neighbours), tout en optimisant les données d'apprentissage.



FEATURE ENGINEERING

Pour donner du sens à notre Modèle, enrichissons-le avec des données métiers utiles à la prise de décision

- *Le taux d'endettement ne doit pas dépasser

selon la banque 33% ou maximum 35%

- * Si possible calculez le reste à vivre (nous n'avons que le total des crédits, mais pas le total des ressources

, ni le total des dépenses nous pouvons utiliser le total des crédits)

si un demandeur a trop de prêts

il pourrait avoir des difficultés financières.

- * Vérifier si sur tous les crédits passés il y a eu des incidents de paiement ou non

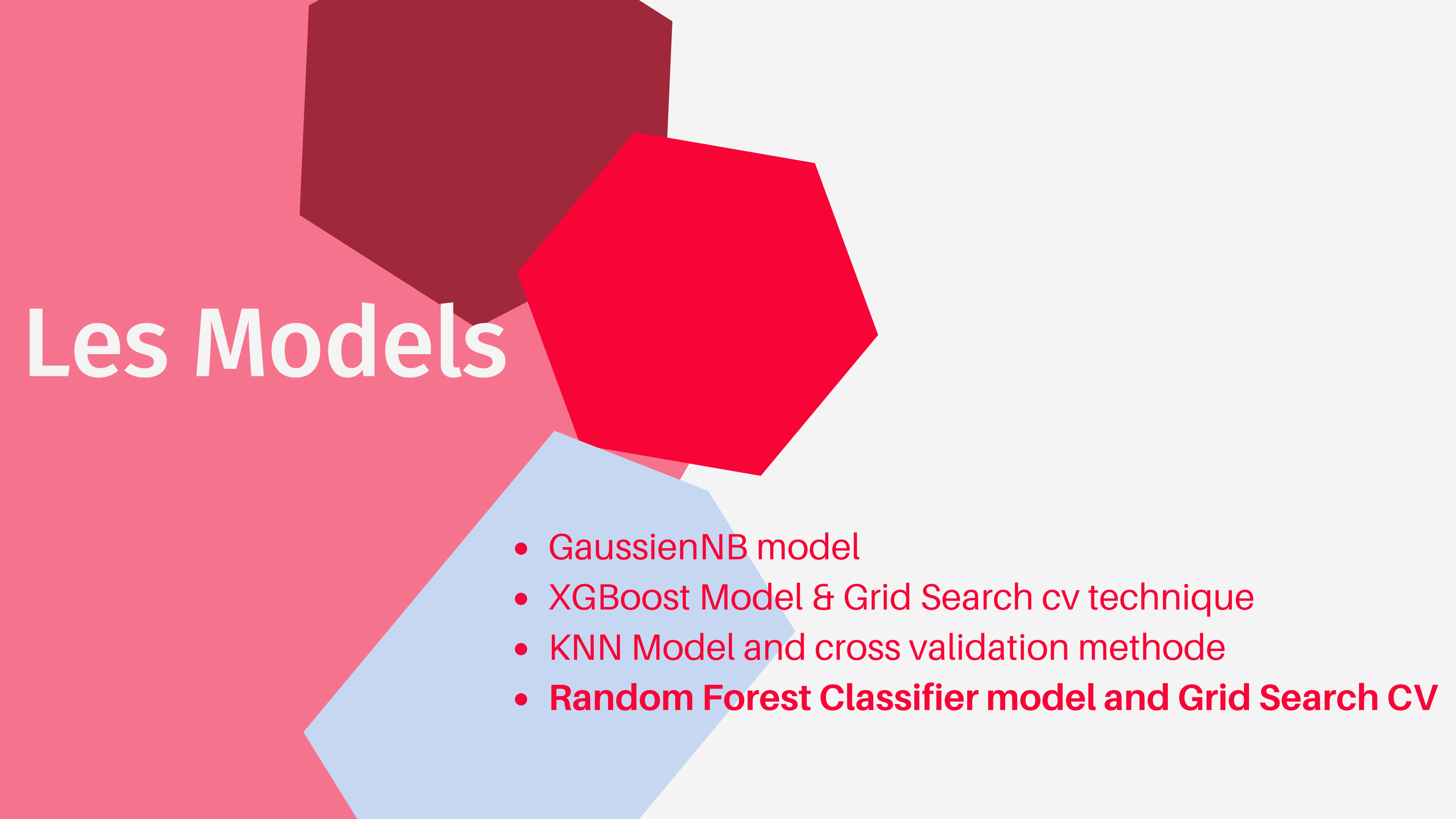
- * Et le montant que l'emprunteur doit payer à la banque au moment du défaut

- * Total des défauts de paiement

- *Nombre de fois que l'emprunteur a été

en retard de 30 à 59 jours, mais pas pire au cours des 2 dernières années.





Les Models

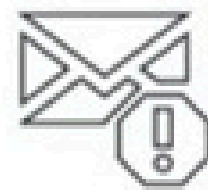
- GaussienNB model
- XGBoost Model & Grid Search cv technique
- KNN Model and cross validation methode
- **Random Forest Classifier model and Grid Search CV**

Choix de la metrique

Notre But est de privilegier la certitude de predire
d

$$F_1 \text{ SCORE} = 2 \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$F_{\beta} \text{ SCORE} = \frac{(1+\beta^2) \text{Precision} * \text{Recall}}{\beta^2 * \text{Precision} + \text{Recall}}$$



PRECISION

F_{0.5} SCORE

F₁ SCORE

F₂ SCORE



RECALL

Fbeta-measure est une métrique configurable à score unique pour évaluer un modèle de classification binaire basé sur les prédictions faites pour la classe positive.

La mesure Fbeta est calculée en utilisant la précision et le rappel.

Fbeta-measure fournit une version configurable de la mesure F pour accorder

La mesure Fbeta est une généralisation de la mesure F qui ajoute un paramètre

de configuration appelé bêta. Une valeur bêta par défaut est 1,0,

ce qui est identique à la mesure F. Une valeur bêta us petite, telle que 0,5, donne plus de poids à la précision et moins au recall

tandis qu'une valeur bêta plus grande, telle que 2,0, donne moins de poids à la précision et plus de poids au rappel ou recall dans le calcul du score.

Re équilibrage des données

Trois méthodes Possibles

Sous-échantillonnage

le processus consistant à sélectionner au hasard les observations de la classe majoritaire et à les retirer de l'ensemble de données d'entraînement. Nous répétons ce processus jusqu'à ce que la distribution de classe souhaitée soit atteinte

Suréchantillonnage

le processus de duplication aléatoire des observations de la classe minoritaire et de leur ajout à l'ensemble de données d'entraînement. Les observations des données d'entraînement sont sélectionnées au hasard avec le remplacement des données

Désavantage:

- Cela peut augmenter la probabilité de surajustement car il reproduit les observations de la classe minoritaire.
- Nous pouvons perdre des informations utiles pendant le processus de sous-échantillonnage.

SMOTE

signifie Synthetic Minority Oversampling Technique. Cette technique est utilisée pour suréchantillonner les données des classes minoritaires. Cette technique évite le surajustement qui se produit dans notre modèle lorsque des observations en double de classes minoritaires sont ajoutées à l'ensemble de données principal (dans la technique de rééchantillonnage)

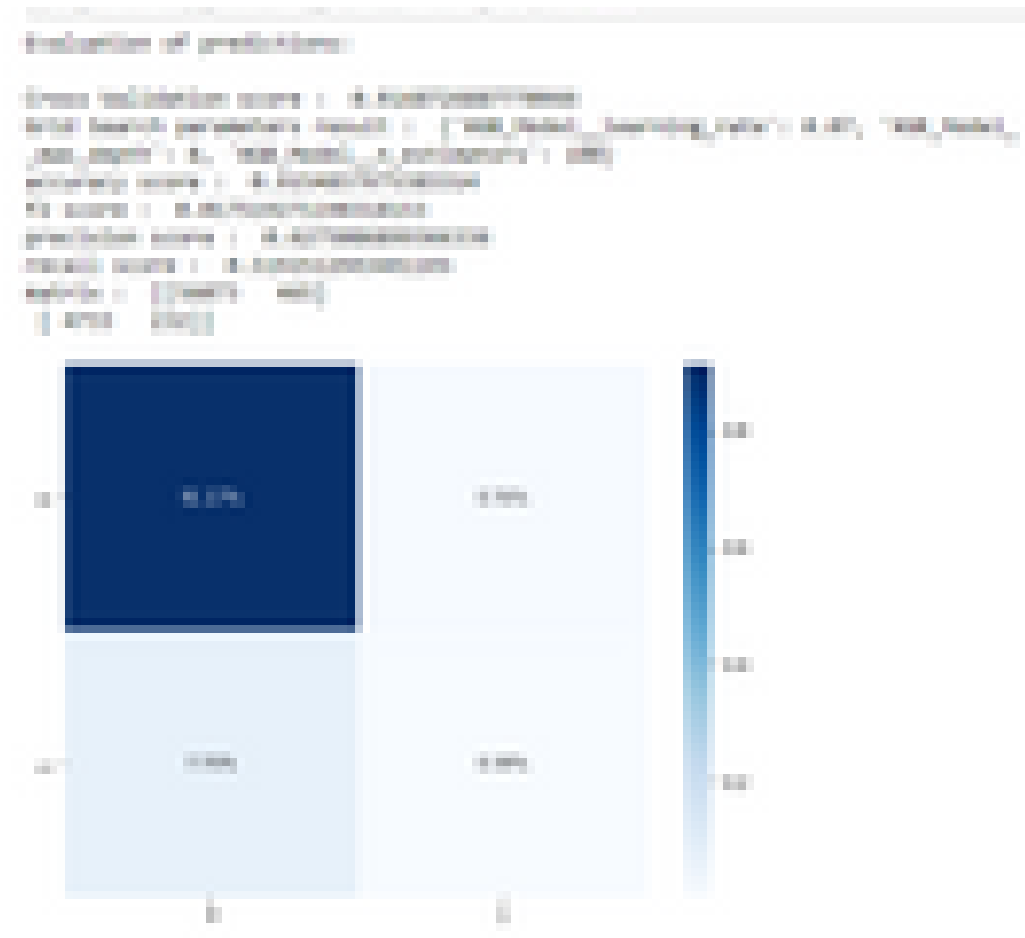
Désavantage:

- Cette technique n'est pas très efficace pour les données de haute dimension.

Avantage:

- Il atténue le risque de surajustement causé par un suréchantillonnage aléatoire car il génère des observations synthétiques plutôt que de dupliquer les observations existantes.
- Aucune information utile n'est perdue dans le processus.

GaussienNB model



```

In [100]: from sklearn.model_selection import ShuffleSplit
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV
from sklearn import svm
from sklearn.model_selection import learning_curve, validation_curve
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
import numpy as np

from sklearn import datasets
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler

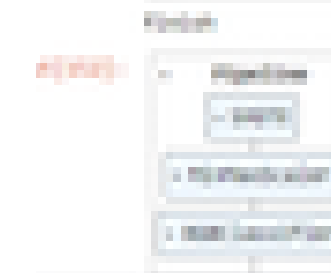
# Create the pipeline
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', svm.SVC())
])

# Create the parameter grid
grid = ParameterGrid([
    ('svm__kernel', ['linear', 'rbf', 'poly']),
    ('svm__gamma', [0.001, 0.01, 0.1, 1, 10]),
    ('svm__C', [0.001, 0.01, 0.1, 1, 10])
])

# Perform the grid search
grid_search = GridSearchCV(pipe, grid, cv=ShuffleSplit(1000, 10, 1))
grid_search.fit(X_train, y_train)

# Print the best parameters
print(grid_search.best_params_)

```



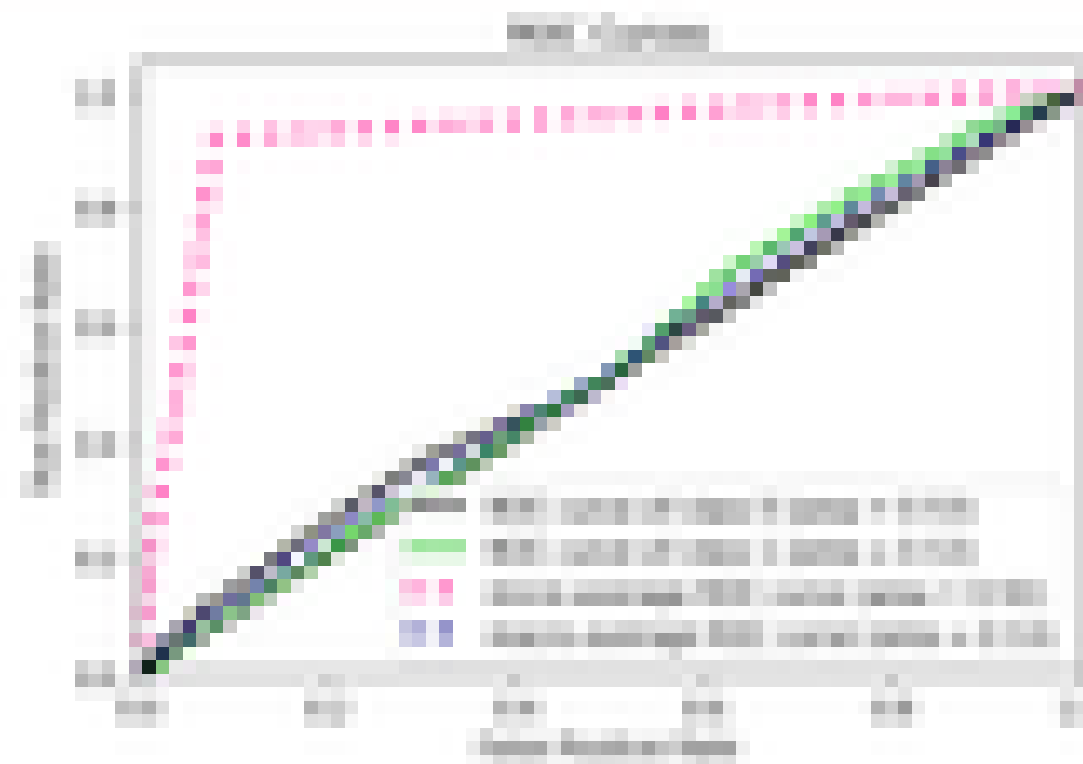
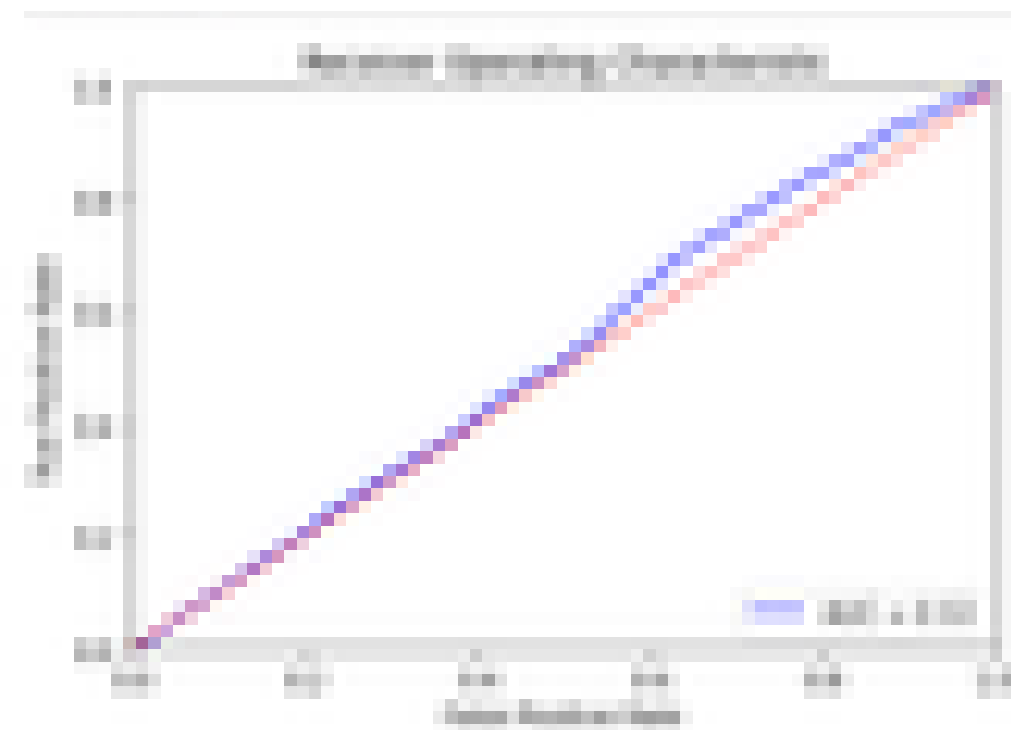
GaussienNB model courbe ROC

Représente la sensibilité en fonction de la spécificité pour toutes les valeurs seuils possibles du marqueur étudié

la sensibilité est la capacité à bien détecter bons payeurs

La spécificité est la capacité du test à bien détecter les mauvais payeurs

Sans Pipeline & kfold



Notre AUC à 50% indique que le marqueur est non informatif

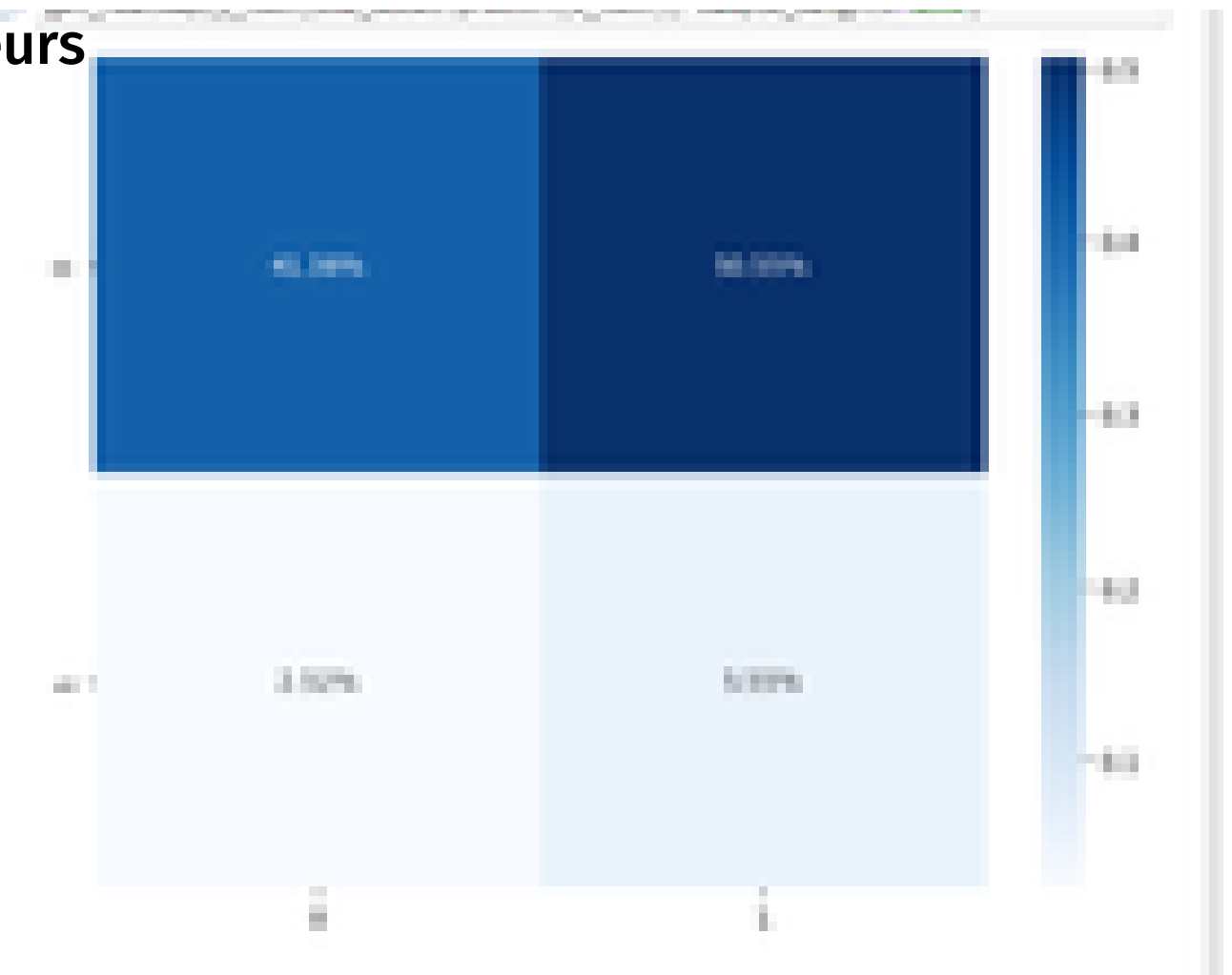
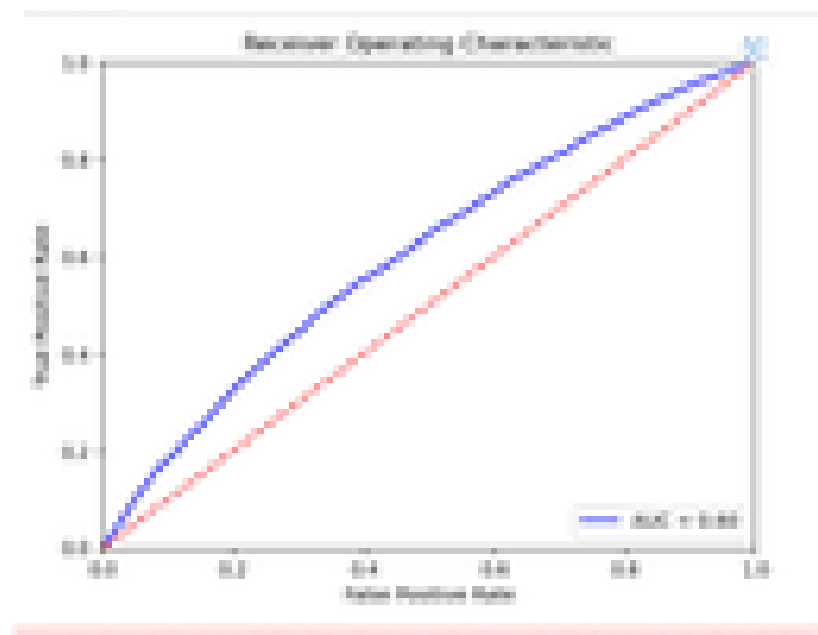
GaussienNB model courbe ROC

Représente la sensibilité en fonction de la spécificité pour toutes les valeurs seuils possibles du marqueur étudié

la sensibilité est la capacité à bien détecter bons payeurs

La spécificité est la capacité du test à bien détecter les mauvais payeurs

Avec Pipeline & kfold



Nous pouvons constater une nette amélioration des résultats

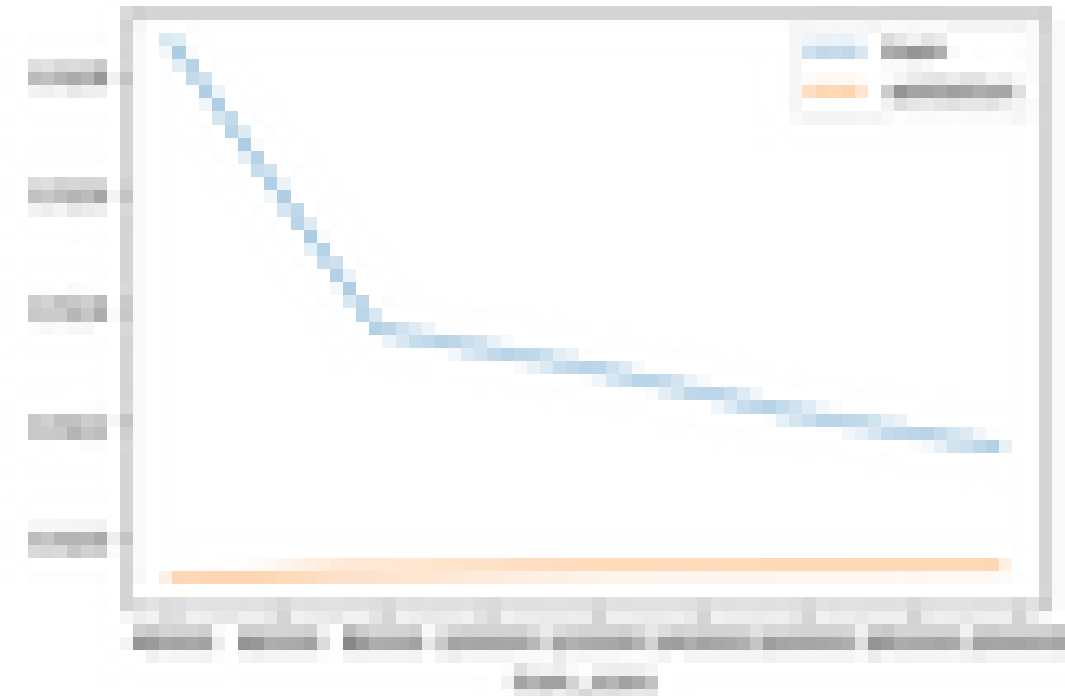
GaussienNB model

Courbe de validation

L'interprétation des résultats d'une courbe de validation peut parfois être délicate. Gardez les points suivants à l'esprit lorsque vous regardez une courbe de validation :

- Idéalement, nous voudrions que la courbe de validation et la courbe d'entraînement se ressemblent le plus possible.
- Si les deux scores sont faibles, le modèle est susceptible d'être sous-adapté. Cela signifie soit que le modèle est trop simple, soit qu'il est informé par trop peu de fonctionnalités. Il se peut aussi que le modèle soit trop régularisé.
- Si la courbe d'entraînement atteint un score élevé relativement rapidement et que la courbe de validation est à la traîne, le modèle est surajustable. Cela signifie que le modèle est très complexe et qu'il y a trop peu de données; ou cela pourrait simplement signifier qu'il y a trop peu de données.
- Nous voudrions la valeur du paramètre où les courbes d'entraînement et de validation sont les plus proches l'une de l'autre.

Une courbe de validation est utilisée pour évaluer un modèle existant en fonction d'hyper-paramètres et n'est pas utilisée pour régler un modèle.



Notre AUC a 50% indique que le marqueur est non informatif

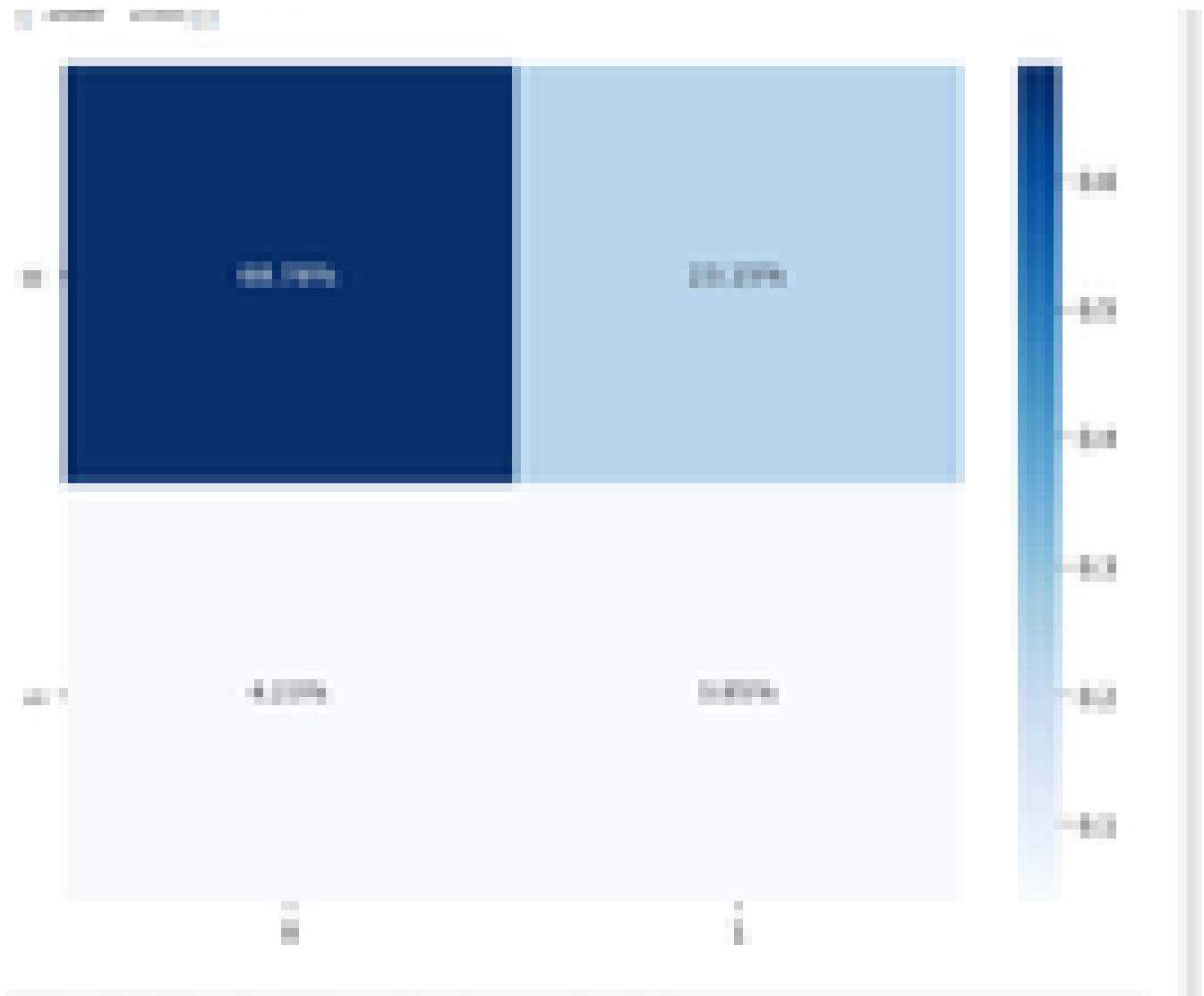
XGBoost Model & Grid Search cv technique

Sans Pipeline & KFold



XGBoost Model & Grid Search cv technique

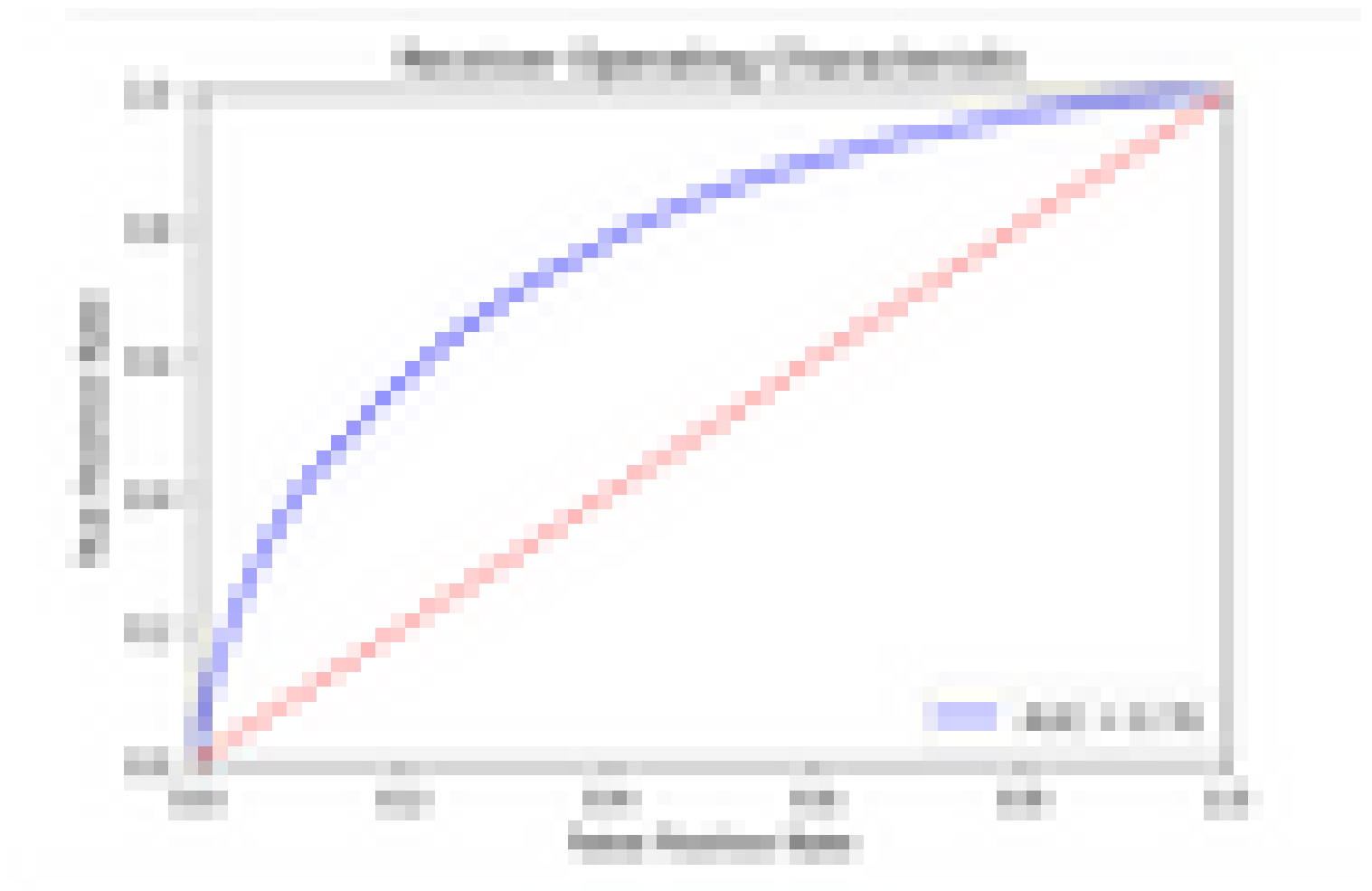
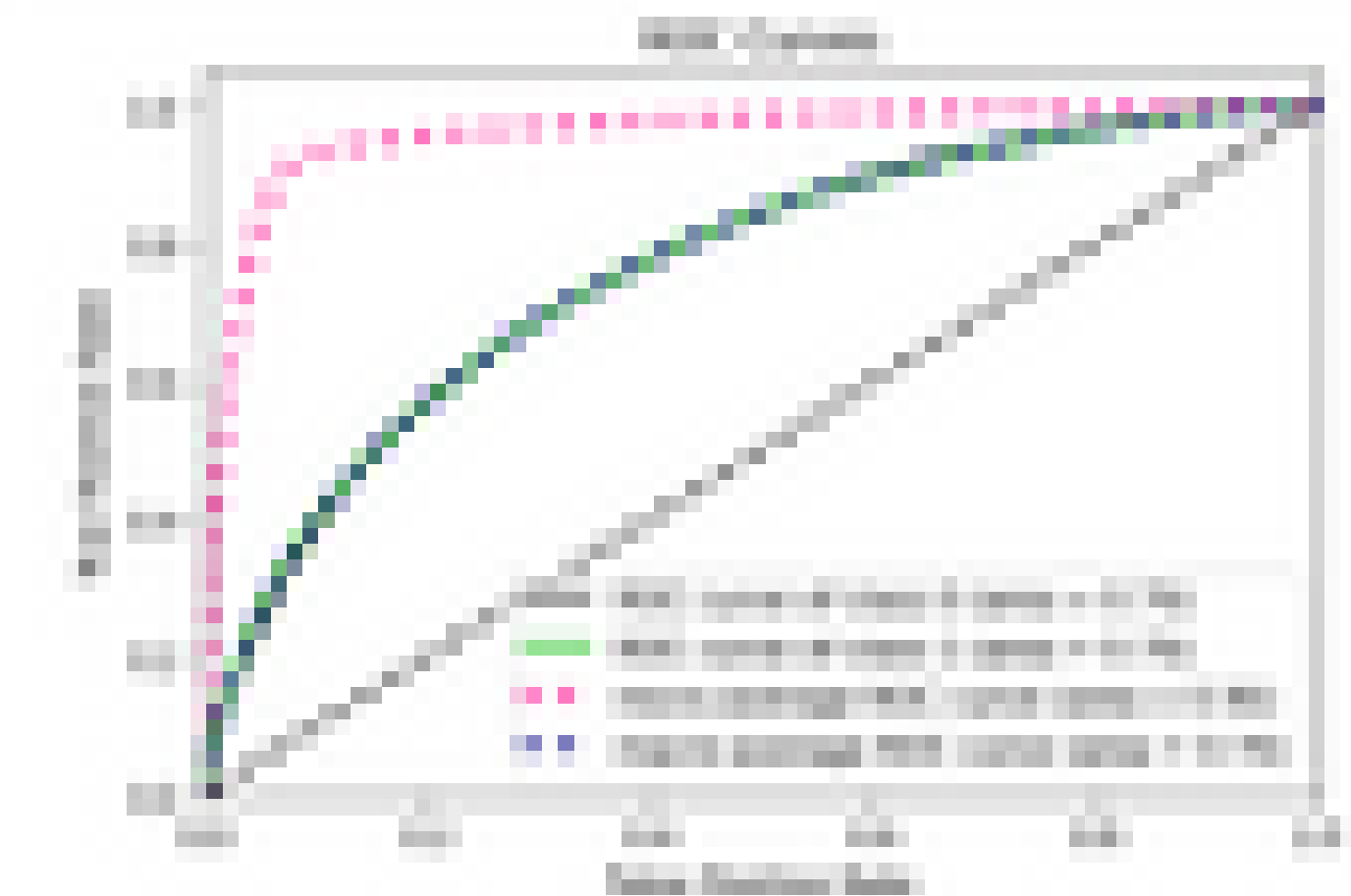
Pipeline & KFold



```
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import log_loss
from sklearn.metrics import brier_score_function
from sklearn.metrics import jaccard_index_score
from sklearn.metrics import hamming_loss
from sklearn.metrics import cohen_kappa_score
from sklearn.metrics import matthews_correlation_coefficient
from sklearn.metrics import average_precision_score
from sklearn.metrics import ndcg_score
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import explained_variance_score
from sklearn.metrics import silhouette_score
from sklearn.metrics import davies_bouldin_score
from sklearn.metrics import calinski_harabasz_score
from sklearn.metrics import adjusted_rand_score
from sklearn.metrics import adjusted_mutual_info_score
from sklearn.metrics import adjusted_rand_score4
from sklearn.metrics import adjusted_rand_score5
from sklearn.metrics import adjusted_rand_score6
from sklearn.metrics import adjusted_rand_score7
from sklearn.metrics import adjusted_rand_score8
from sklearn.metrics import adjusted_rand_score9
from sklearn.metrics import adjusted_rand_score10
from sklearn.metrics import adjusted_rand_score11
from sklearn.metrics import adjusted_rand_score12
from sklearn.metrics import adjusted_rand_score13
from sklearn.metrics import adjusted_rand_score14
from sklearn.metrics import adjusted_rand_score15
from sklearn.metrics import adjusted_rand_score16
from sklearn.metrics import adjusted_rand_score17
from sklearn.metrics import adjusted_rand_score18
from sklearn.metrics import adjusted_rand_score19
from sklearn.metrics import adjusted_rand_score20
from sklearn.metrics import adjusted_rand_score21
from sklearn.metrics import adjusted_rand_score22
from sklearn.metrics import adjusted_rand_score23
from sklearn.metrics import adjusted_rand_score24
from sklearn.metrics import adjusted_rand_score25
from sklearn.metrics import adjusted_rand_score26
from sklearn.metrics import adjusted_rand_score27
from sklearn.metrics import adjusted_rand_score28
from sklearn.metrics import adjusted_rand_score29
from sklearn.metrics import adjusted_rand_score30
from sklearn.metrics import adjusted_rand_score31
from sklearn.metrics import adjusted_rand_score32
from sklearn.metrics import adjusted_rand_score33
from sklearn.metrics import adjusted_rand_score34
from sklearn.metrics import adjusted_rand_score35
from sklearn.metrics import adjusted_rand_score36
from sklearn.metrics import adjusted_rand_score37
from sklearn.metrics import adjusted_rand_score38
from sklearn.metrics import adjusted_rand_score39
from sklearn.metrics import adjusted_rand_score40
from sklearn.metrics import adjusted_rand_score41
from sklearn.metrics import adjusted_rand_score42
from sklearn.metrics import adjusted_rand_score43
from sklearn.metrics import adjusted_rand_score44
from sklearn.metrics import adjusted_rand_score45
from sklearn.metrics import adjusted_rand_score46
from sklearn.metrics import adjusted_rand_score47
from sklearn.metrics import adjusted_rand_score48
from sklearn.metrics import adjusted_rand_score49
from sklearn.metrics import adjusted_rand_score50
from sklearn.metrics import adjusted_rand_score51
from sklearn.metrics import adjusted_rand_score52
from sklearn.metrics import adjusted_rand_score53
from sklearn.metrics import adjusted_rand_score54
from sklearn.metrics import adjusted_rand_score55
from sklearn.metrics import adjusted_rand_score56
from sklearn.metrics import adjusted_rand_score57
from sklearn.metrics import adjusted_rand_score58
from sklearn.metrics import adjusted_rand_score59
from sklearn.metrics import adjusted_rand_score60
from sklearn.metrics import adjusted_rand_score61
from sklearn.metrics import adjusted_rand_score62
from sklearn.metrics import adjusted_rand_score63
from sklearn.metrics import adjusted_rand_score64
from sklearn.metrics import adjusted_rand_score65
from sklearn.metrics import adjusted_rand_score66
from sklearn.metrics import adjusted_rand_score67
from sklearn.metrics import adjusted_rand_score68
from sklearn.metrics import adjusted_rand_score69
from sklearn.metrics import adjusted_rand_score70
from sklearn.metrics import adjusted_rand_score71
from sklearn.metrics import adjusted_rand_score72
from sklearn.metrics import adjusted_rand_score73
from sklearn.metrics import adjusted_rand_score74
from sklearn.metrics import adjusted_rand_score75
from sklearn.metrics import adjusted_rand_score76
from sklearn.metrics import adjusted_rand_score77
from sklearn.metrics import adjusted_rand_score78
from sklearn.metrics import adjusted_rand_score79
from sklearn.metrics import adjusted_rand_score80
from sklearn.metrics import adjusted_rand_score81
from sklearn.metrics import adjusted_rand_score82
from sklearn.metrics import adjusted_rand_score83
from sklearn.metrics import adjusted_rand_score84
from sklearn.metrics import adjusted_rand_score85
from sklearn.metrics import adjusted_rand_score86
from sklearn.metrics import adjusted_rand_score87
from sklearn.metrics import adjusted_rand_score88
from sklearn.metrics import adjusted_rand_score89
from sklearn.metrics import adjusted_rand_score90
from sklearn.metrics import adjusted_rand_score91
from sklearn.metrics import adjusted_rand_score92
from sklearn.metrics import adjusted_rand_score93
from sklearn.metrics import adjusted_rand_score94
from sklearn.metrics import adjusted_rand_score95
from sklearn.metrics import adjusted_rand_score96
from sklearn.metrics import adjusted_rand_score97
from sklearn.metrics import adjusted_rand_score98
from sklearn.metrics import adjusted_rand_score99
from sklearn.metrics import adjusted_rand_score100
```

Amelioration du FBeta score

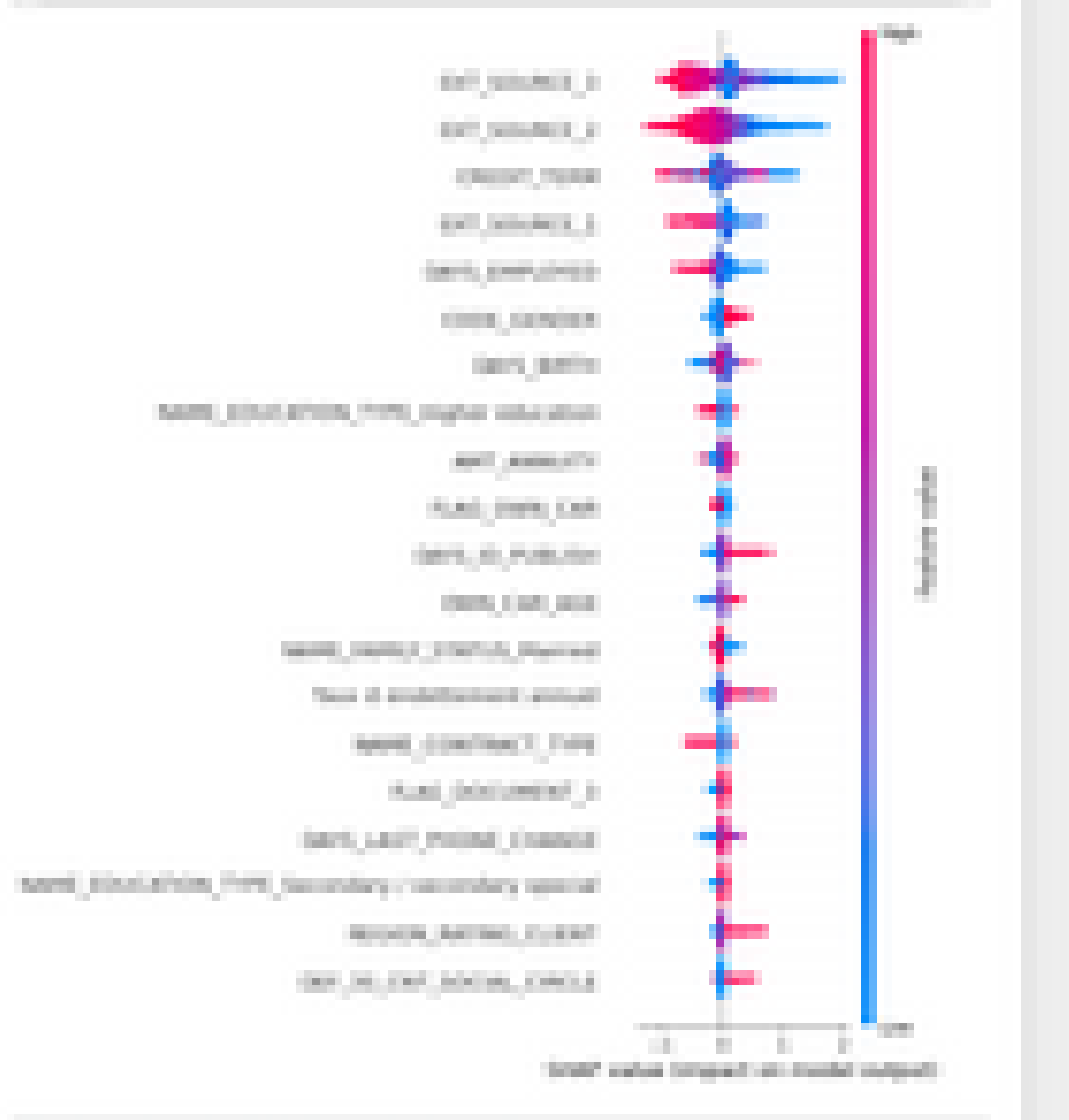
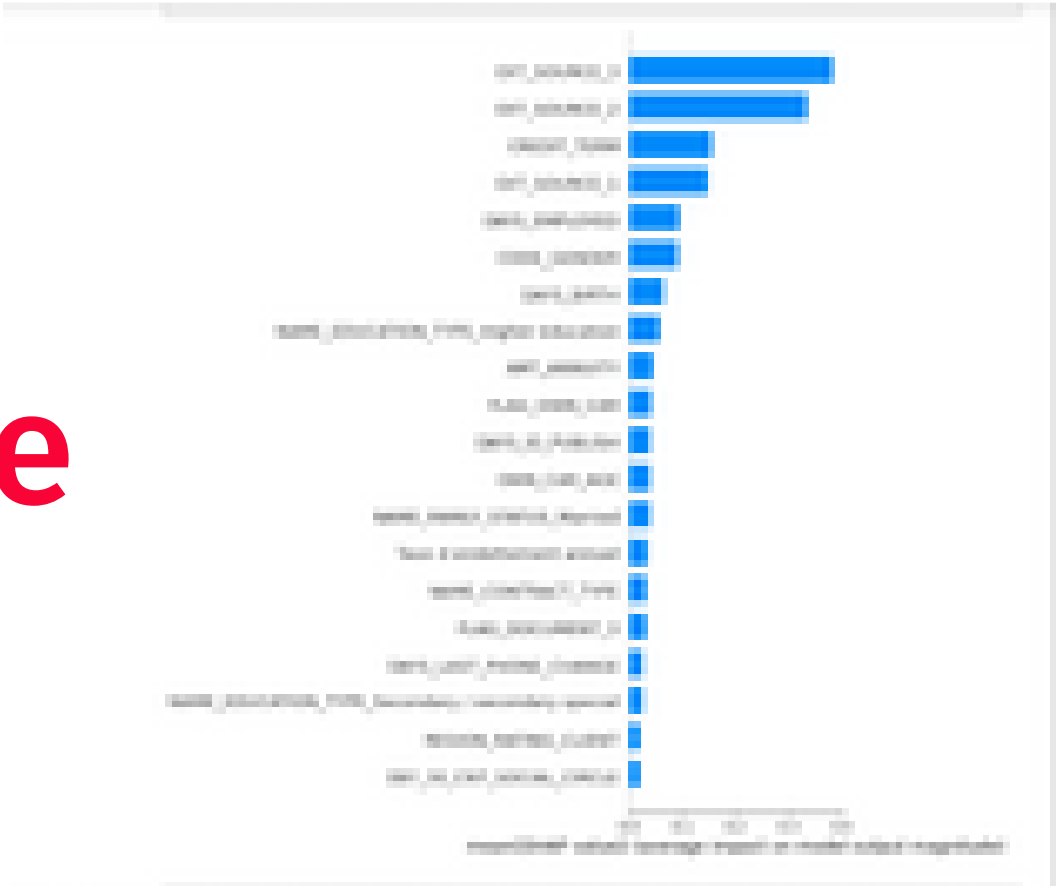
XGBoost Model



Notre model XGBoost represente un meilleur score que le model precedent et apres utilisation du pipeline

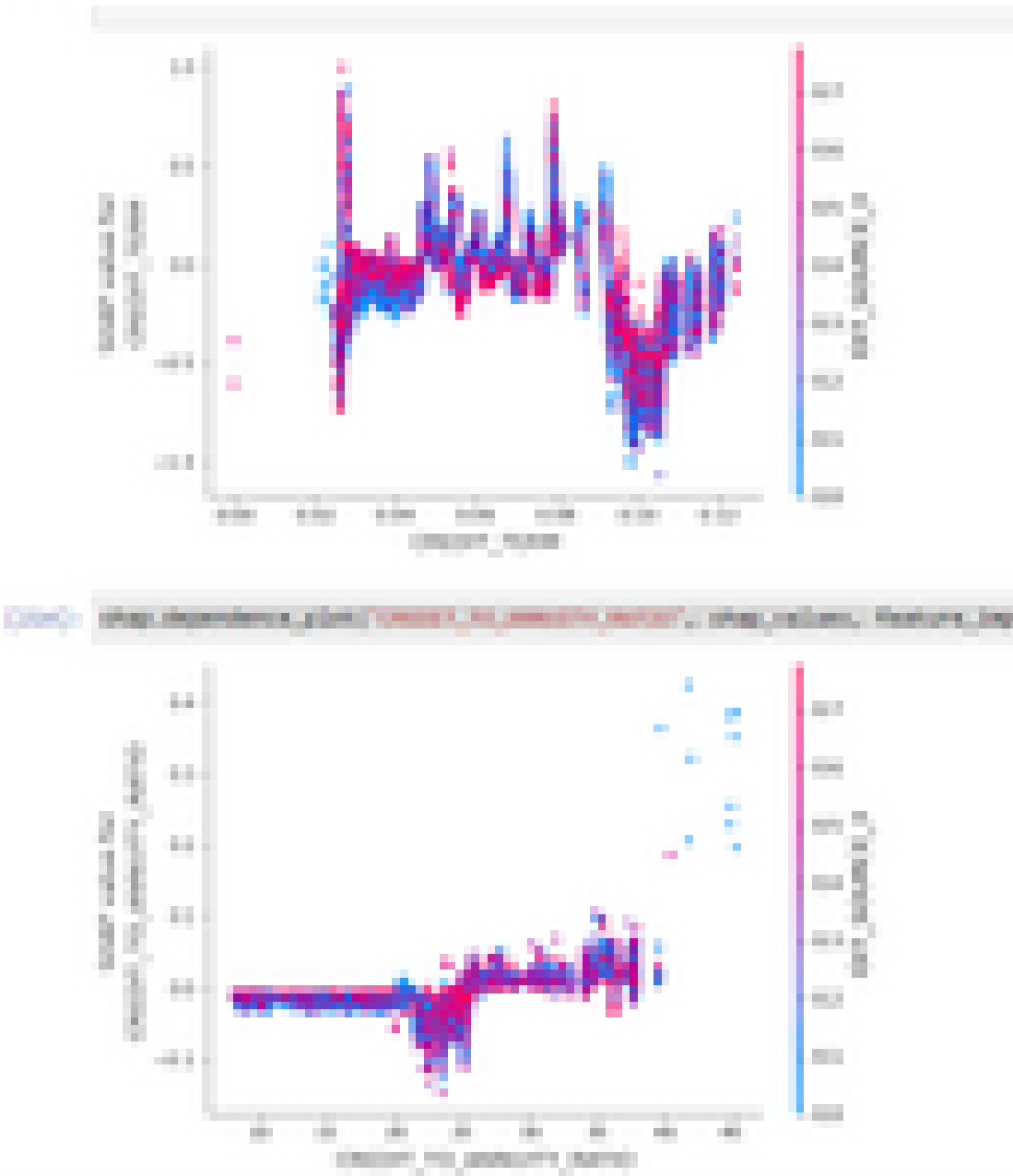
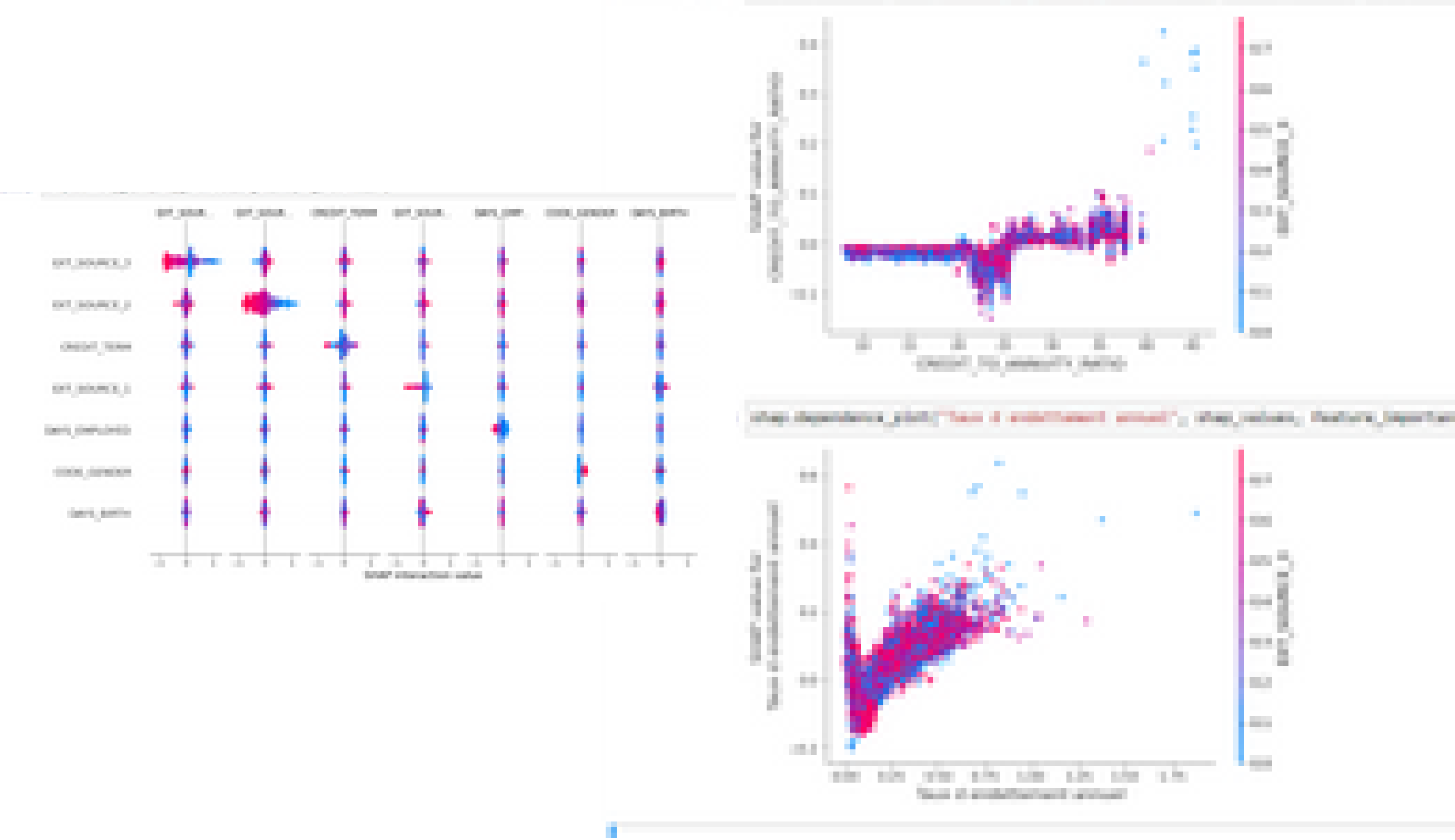
XGBoost Model & Grid Search cv technique

Sans pipeline

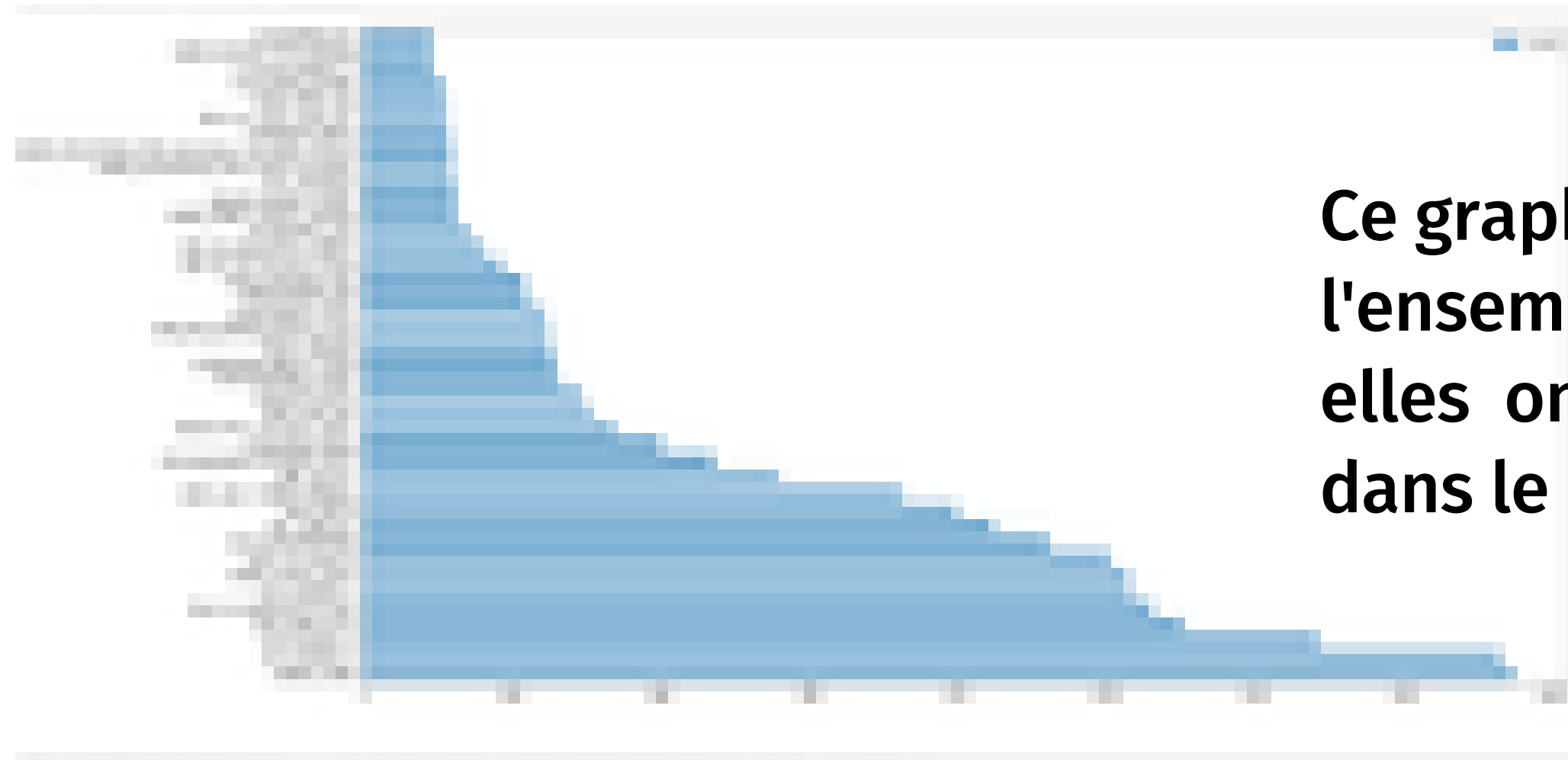


Avec Pipeline

XGBoost Model & Grid Search cv technique



XGBoost Model & Grid Search cv technique



Ce graph montre l'importance de l'ensemble de nos fonctionnalités, elles ont joué un rôle déterminant dans le processus d'apprentissage

XGBoost Model



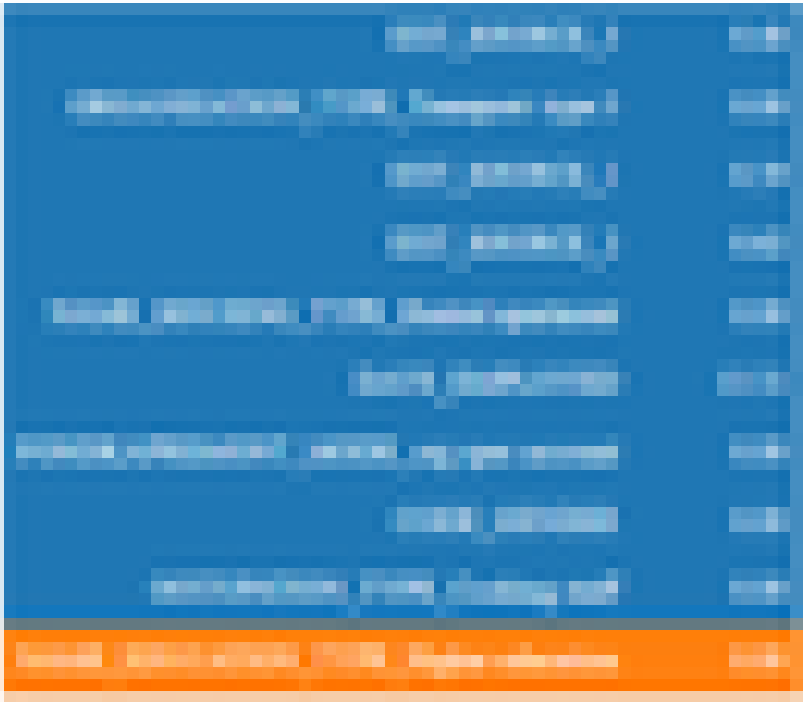
Ce graph montre l'importance de l'ensemble de nos fonctionnalités, elles ont joué un rôle déterminant dans le processus d'apprentissage

XGBoost LIME Analysis

Pour analyser l'importance de nos fonctionnalités de façon local propre à chaque individu, Nous pouvons utiliser l'analyse LIME

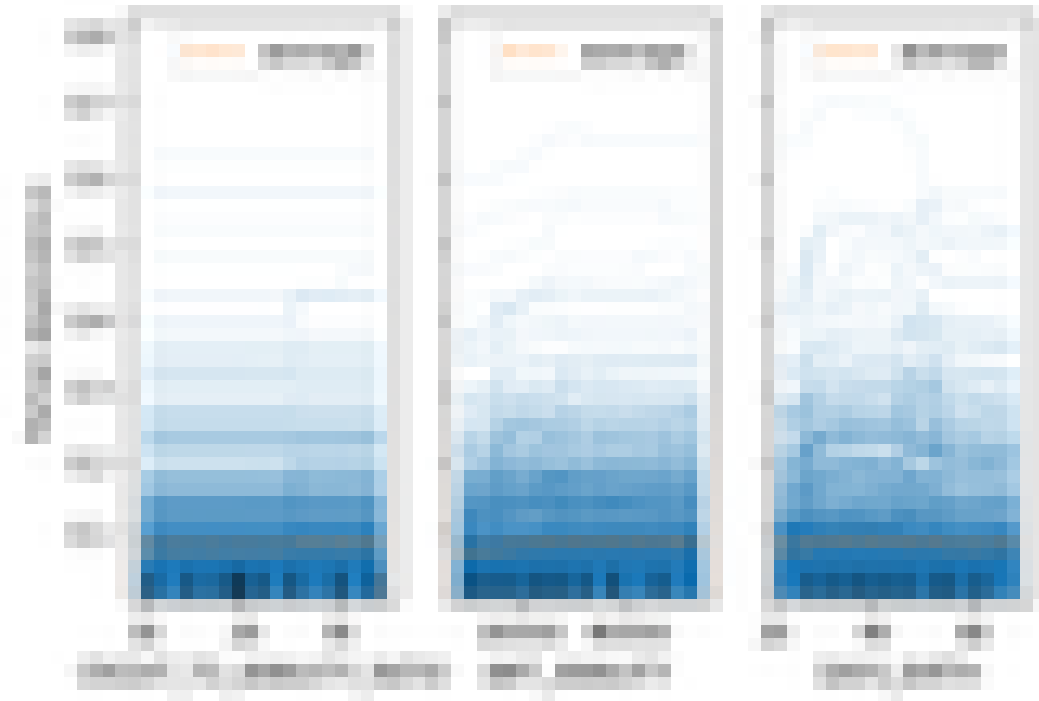
LIME signifie Local Interpretable Model-Agnostic Explanations. LIME est une méthode indépendante du modèle, ce qui signifie qu'elle peut être appliquée à n'importe quel modèle d'apprentissage automatique. La technique tente de comprendre le modèle en perturbant l'entrée d'échantillons de données et en comprenant comment les prédictions (sorties) changent. La sortie de LIME est une liste d'explications, reflétant la contribution de chaque caractéristique à la prédiction d'un échantillon de données. Cela fournit une interprétabilité locale et permet également de déterminer quelles modifications de fonctionnalités auront le plus d'impact sur la prédiction.

Lime indique la valeur variable pour laquelle un candidat est jugé bon candidat tandis que le graph précédent met en valeur les nouvelles fonctionnalités comme étant importante dans la prédiction du modèle



Le graph de lime a réaffirmé l'importance des nos nouvelles caractéristiques notamment Taux d'endettement et credit term et credit term ratio

XGBoost Partial dependence display



Pour analyser l'importance de nos fonctionnalités de façon locale propre à chaque individu, Nous pouvons utiliser l'analyse LIME

Les diagrammes de dépendance partielle (PDP) et les diagrammes d'attente conditionnelle individuelle (ICE) peuvent être utilisés pour visualiser et analyser l'interaction entre la réponse cible [1] et un ensemble de caractéristiques d'entrée d'intérêt.

KNN Model and cross validation methode

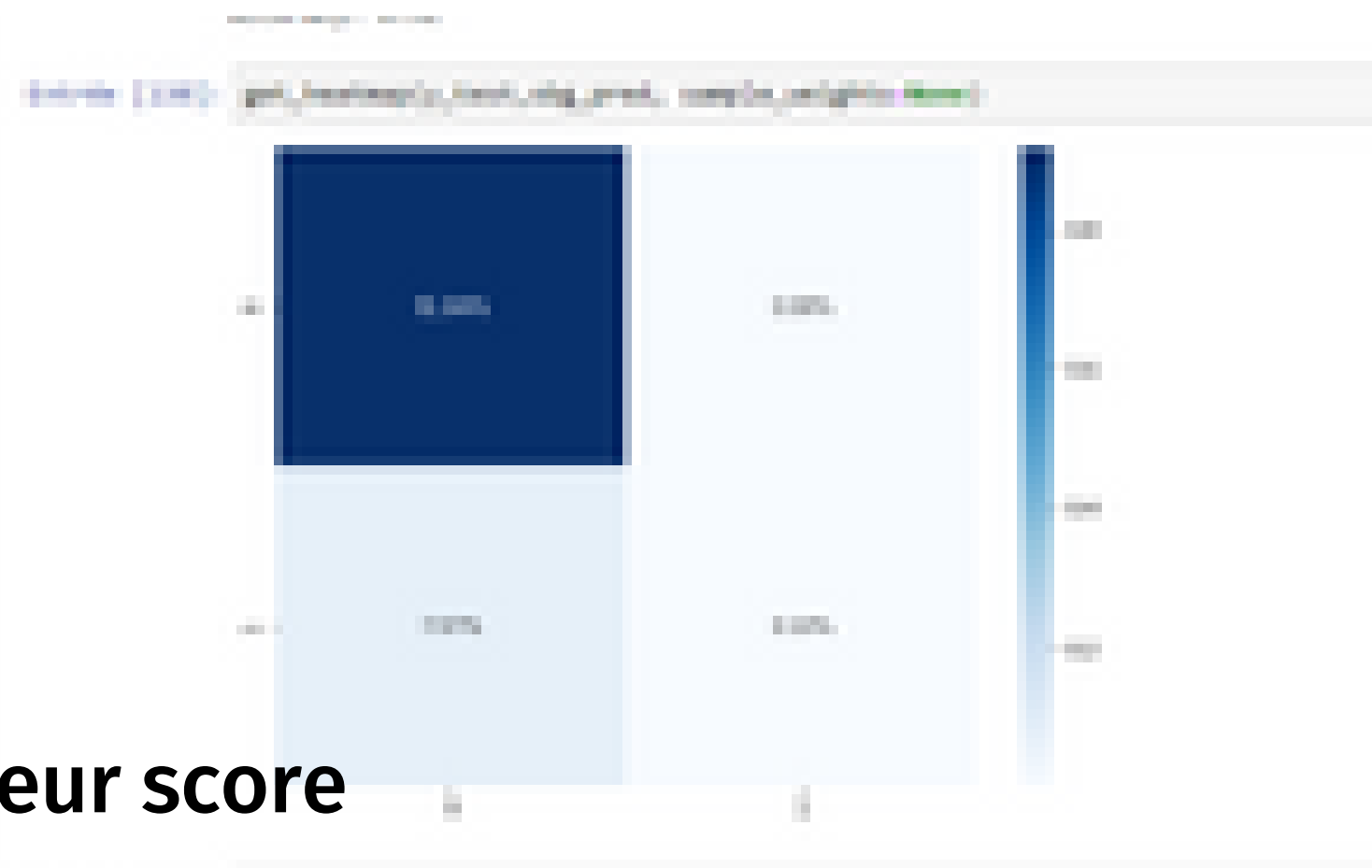


KNN MODEL & Cross Validation

```
[10]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
X_train, y_train, X_test, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
ylg_pred = knn_model.predict(X_test)

#crossing = get_cross_val_score(y_test, ylg_pred, X_test, y_test)

y_test_score = []
for k in range(1,10):
    score = cross_val_score(knn_model(X_train, y_train), X_test, y_test, cv=5, n_jobs=-1)
    y_test_score.append(score)
print(y_test_score)
```



Meilleur hyperparametre pour un meilleur score
est $k = 9$

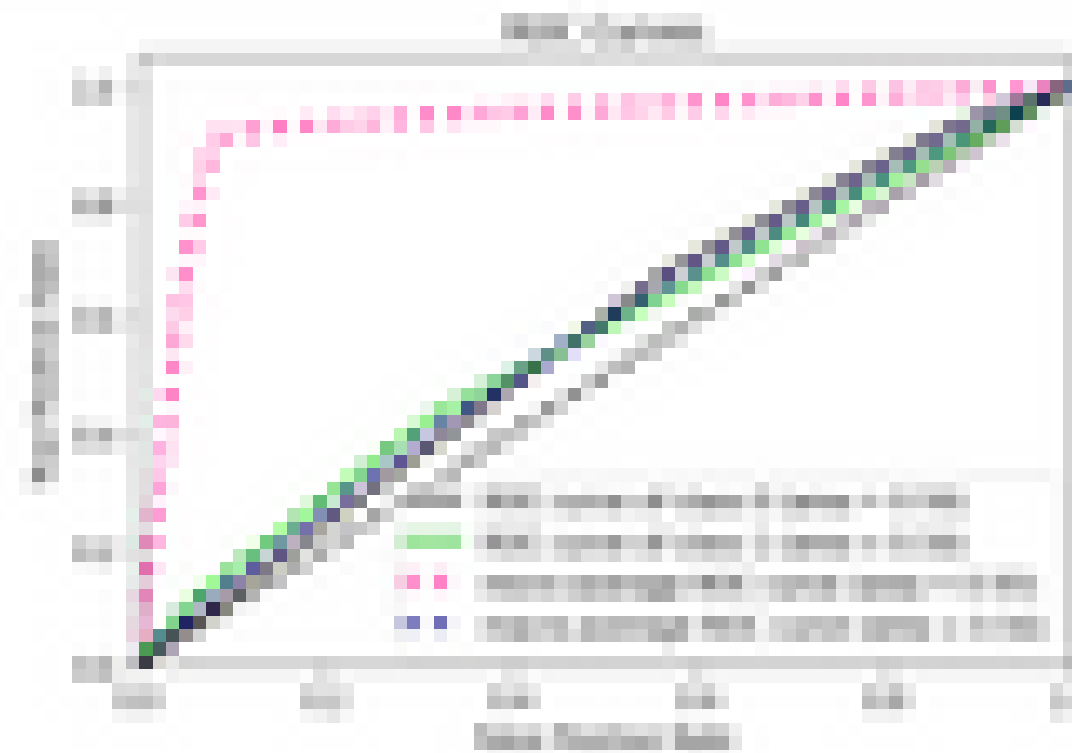
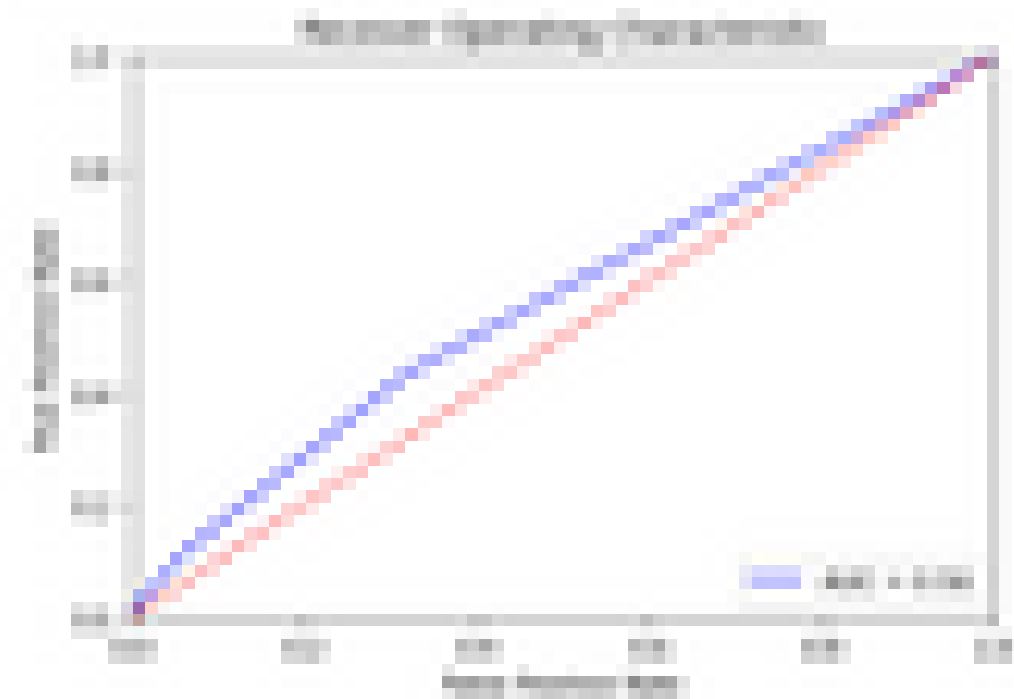
KNN Model and cross validation methode

KNN MODEL & Cross Validation

```
[10]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.cross_validation import cross_val_score
X_train, y_train, X_test, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
knn_pred = knn_model.predict(X_test)

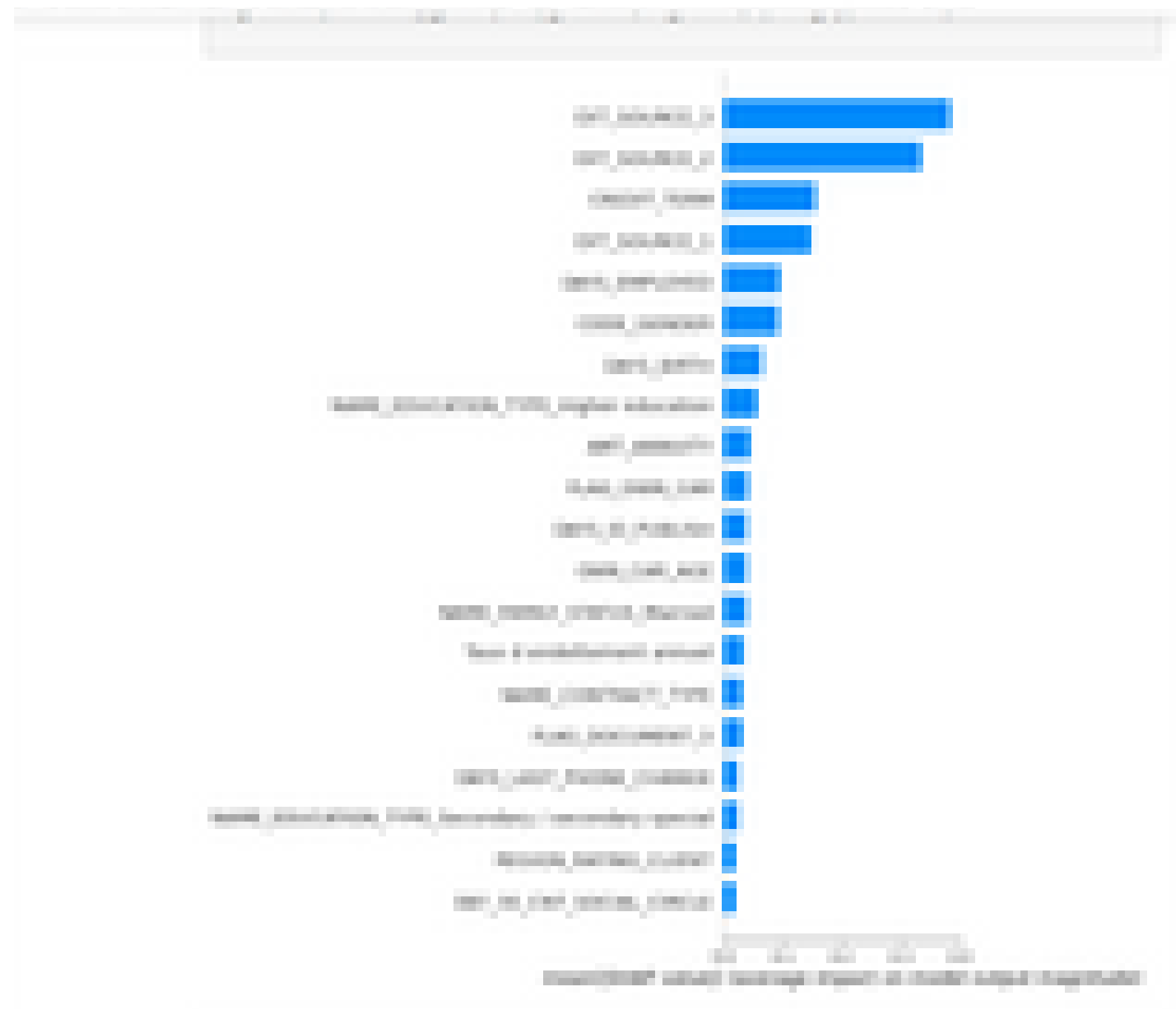
# Scoring
get_f1_score(y_test, knn_pred, average='micro')

val_score = []
for k in range(1,10):
    score = cross_val_score(knn_model(X_train, y_train, n_neighbors=k), X_test, y_test, cv=5)
    val_score.append(score)
plt.plot(val_score)
```

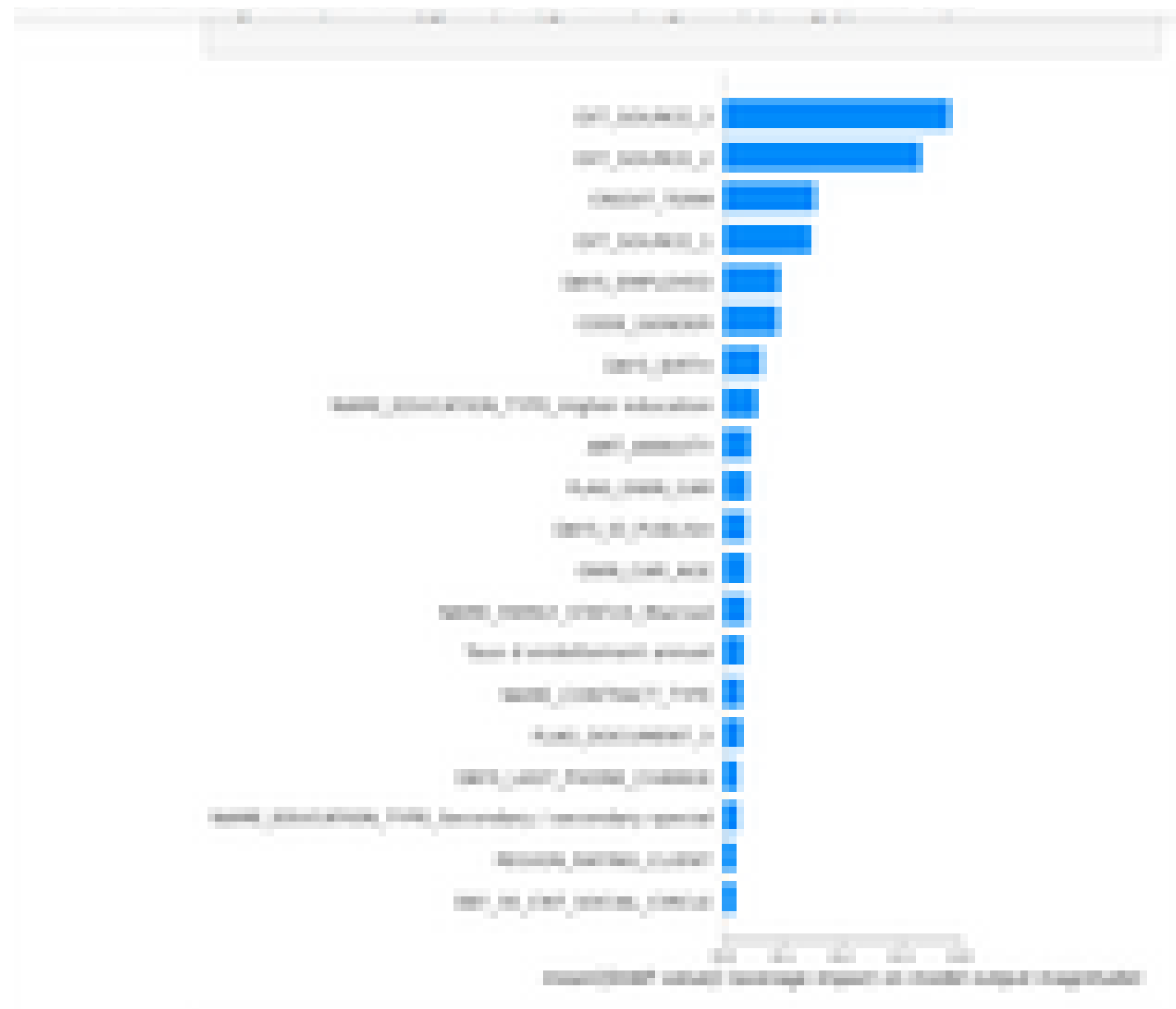


Tout comme les résultat de notre premier modele l'AUC avoisine 50% avec une amélioration en utilisant pipeline

KNN Model and cross validation methode

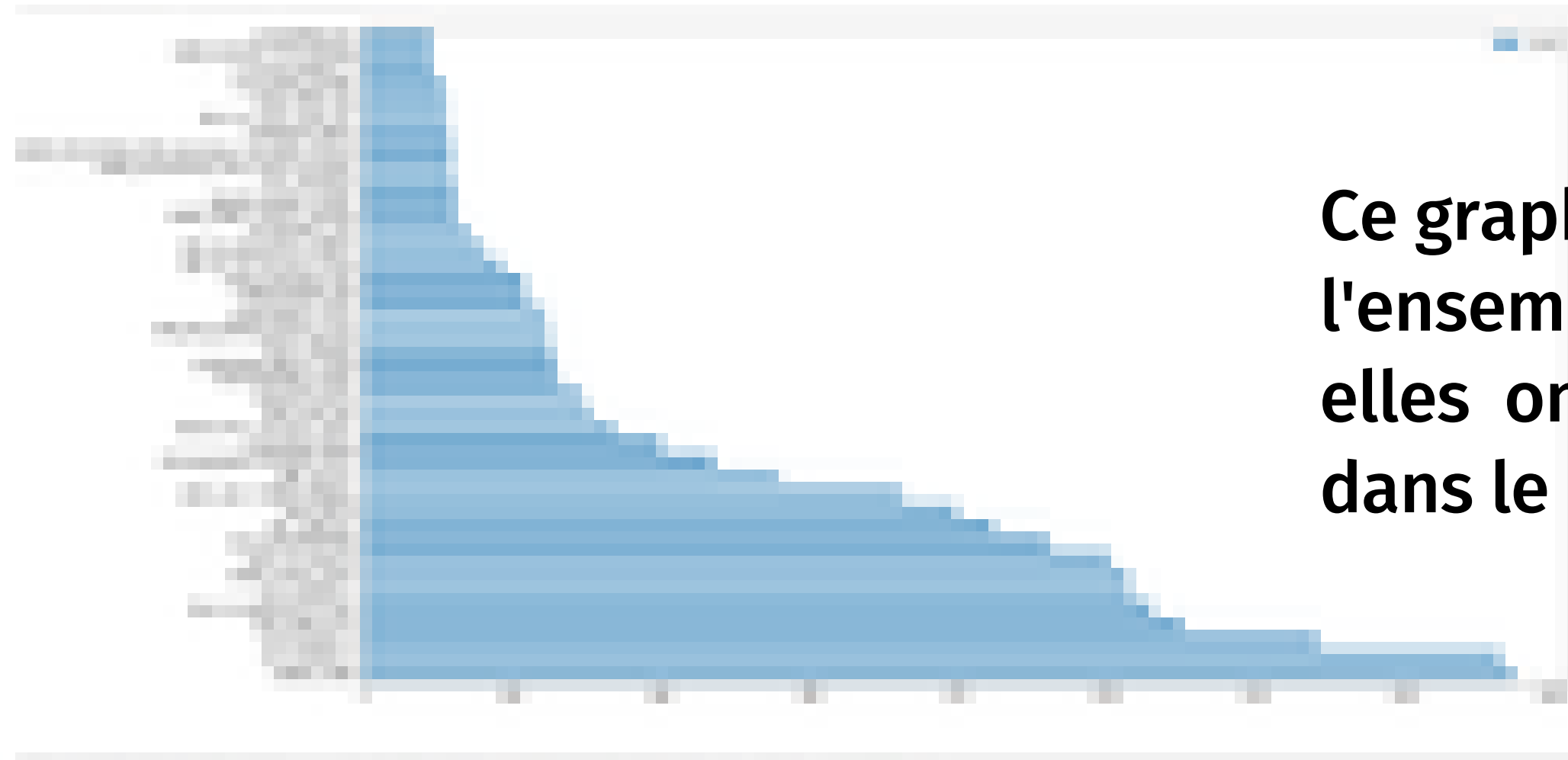


KNN Model and cross validation methode



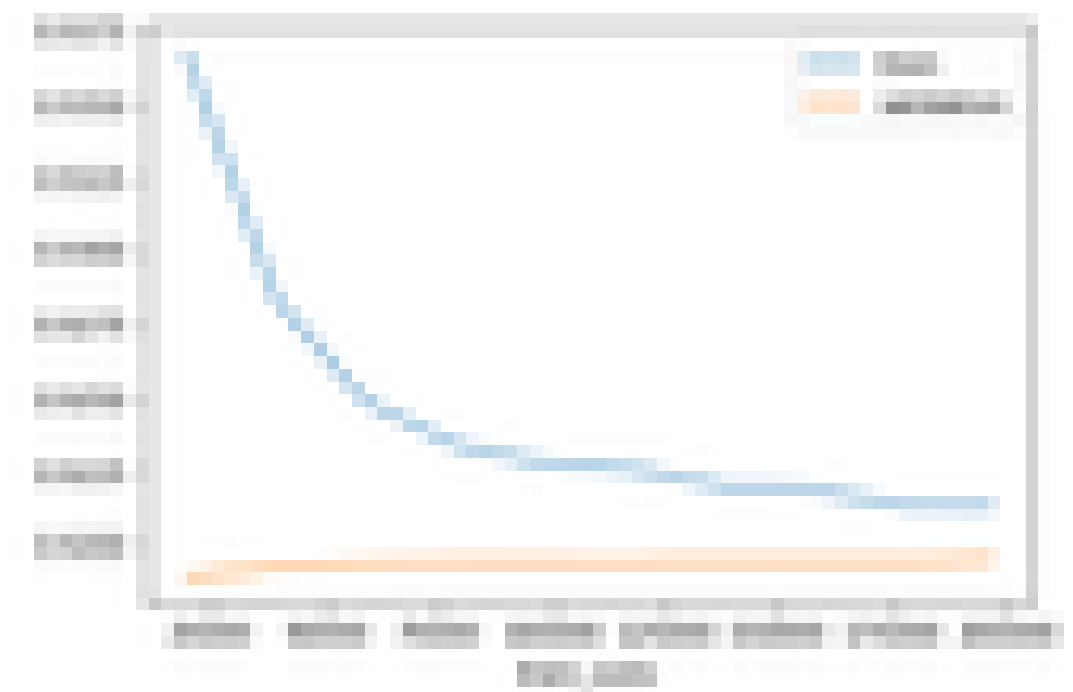
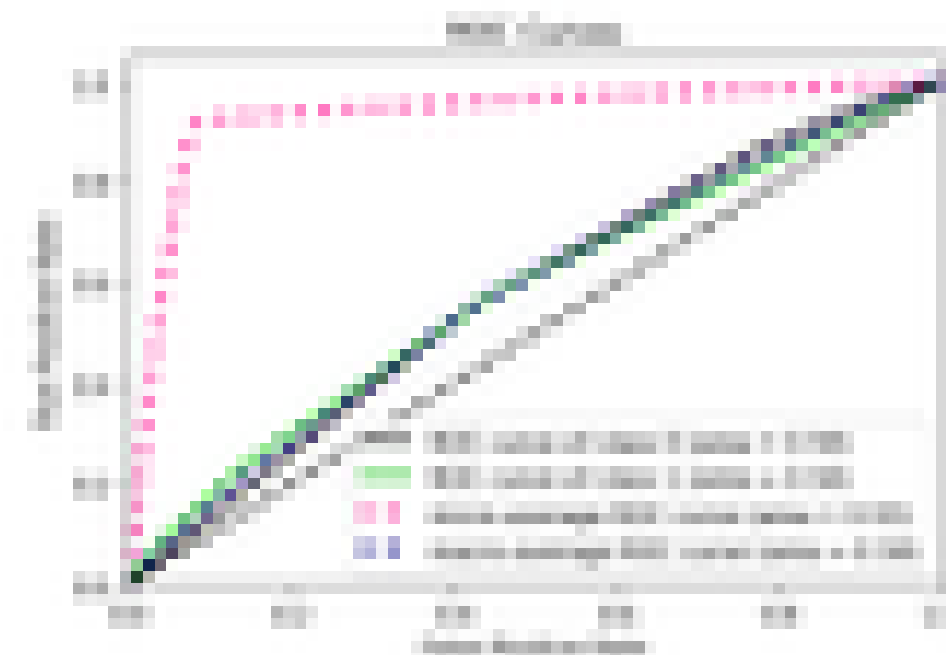
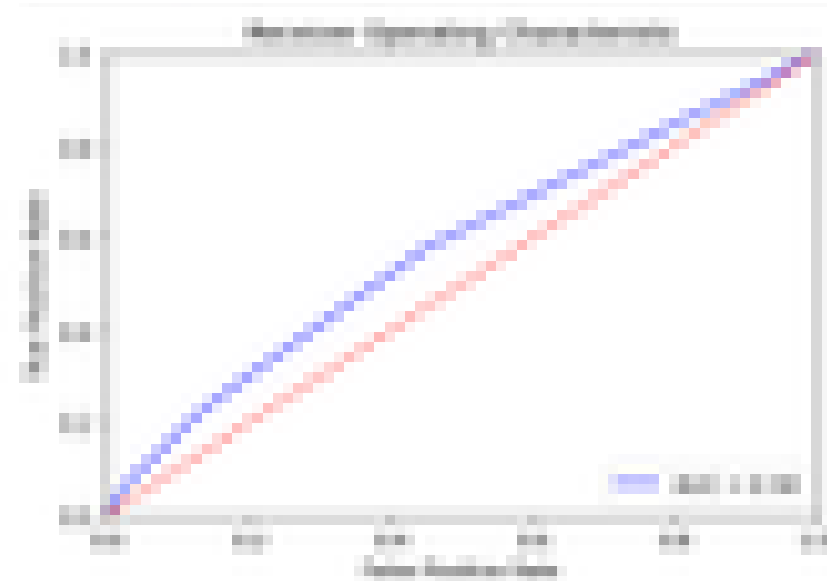
KNN Model

L'importance de la fonctionnalité de
façon global



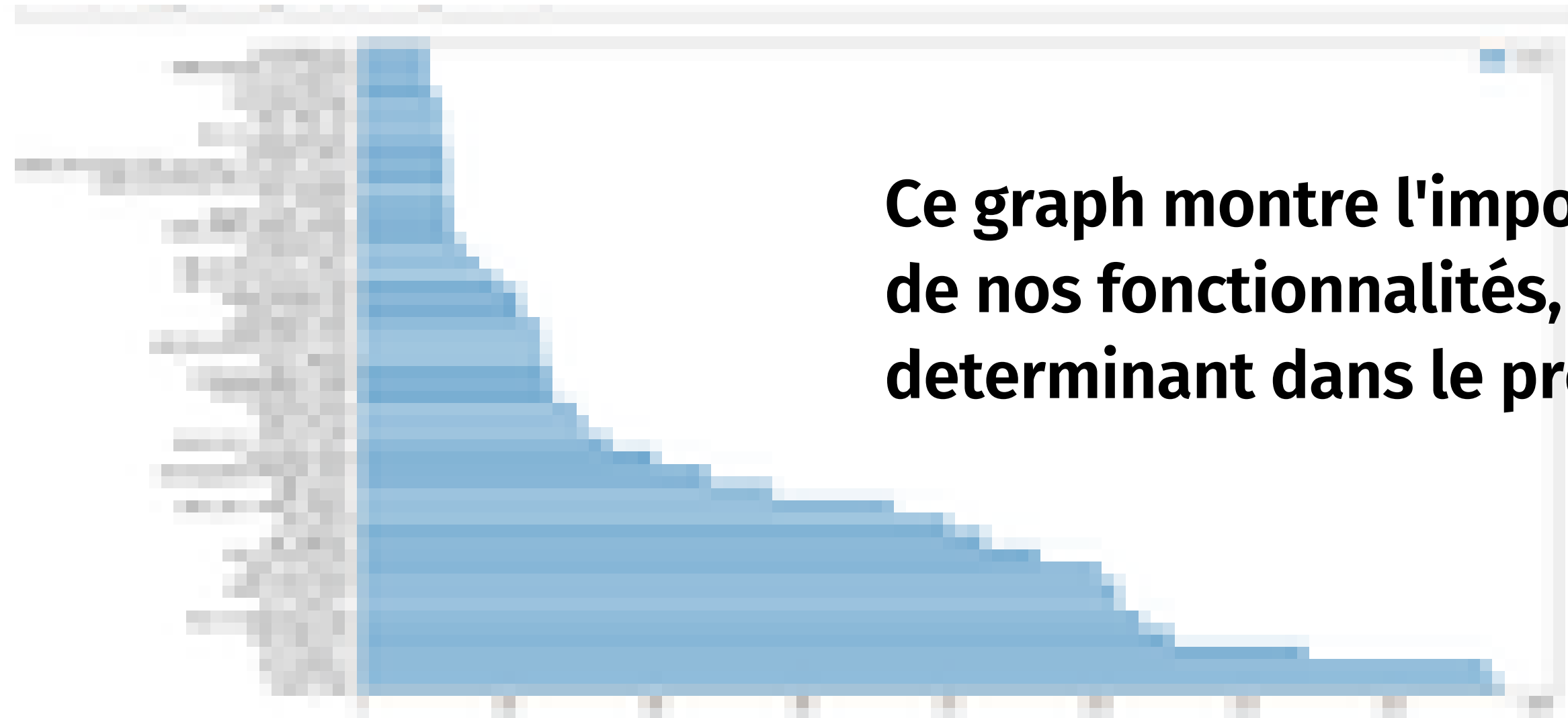
Ce graph montre l'importance de
l'ensemble de nos fonctionnalités,
elles ont joué un rôle déterminant
dans le process d'apprentissage

Random Forest Classifier



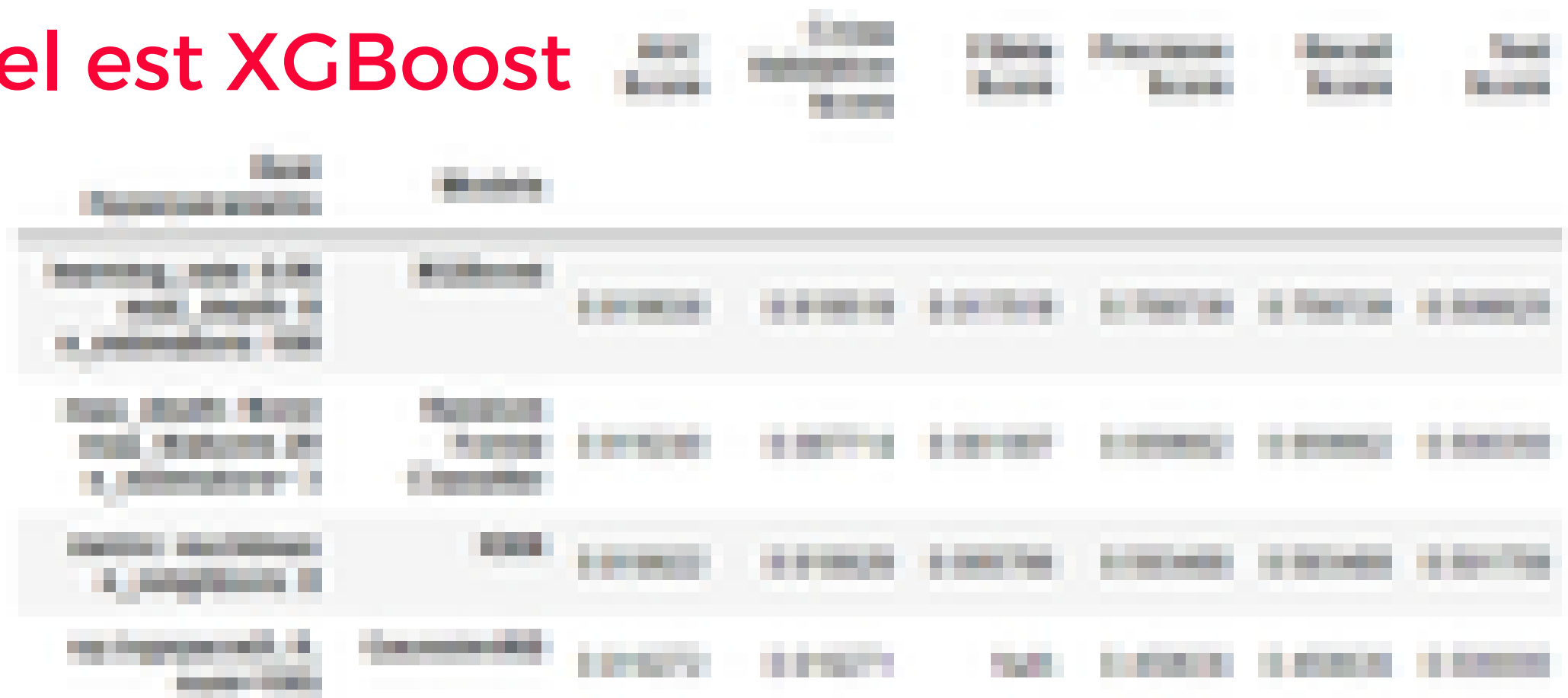
Resultats similaires aux resultat du model KNN ET GBN

Random Forest Classifier



Ce graph montre l'importance de l'ensemble de nos fonctionnalités, elles ont joué un rôle déterminant dans le process d'apprentissage

Le meilleur model est XGBoost



The screenshot displays a Kaggle competition leaderboard. At the top, there are navigation icons for sorting and filtering. The table lists several teams, with 'XGBoost' being the top performer. The columns include team names, scores, and various performance metrics. The 'XGBoost' team is highlighted in the first row.

Team	Score	Rank	Other Metrics
XGBoost	0.95	1	0.95, 0.95, 0.95, 0.95, 0.95, 0.95, 0.95, 0.95
Team 2	0.94	2	0.94, 0.94, 0.94, 0.94, 0.94, 0.94, 0.94, 0.94
Team 3	0.93	3	0.93, 0.93, 0.93, 0.93, 0.93, 0.93, 0.93, 0.93
Team 4	0.92	4	0.92, 0.92, 0.92, 0.92, 0.92, 0.92, 0.92, 0.92
Team 5	0.91	5	0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91

Home Credit

Home Credit

GBM Analysis

XGBClassifier

Is nearest neighbors_k_201

Decision Tree Bayes

Random Forest Classifier

Age

Previous credit

Contract

Age Range

Select a candidate

Is in client

Contract

clients correspond to use filter

HOME
CREDIT



Home Credit Platforms

Customer ID: 123456
Name: John
Age: 35 years

Age

Previous credit

Contract

Age Range

Select a candidate

Is in client

Contract

clients correspond to use filter

HOME
CREDIT



Home Credit Platforms

Customer ID: 123456
Name: John
Age: 35 years

XGBClassifier

Specifications

Age

Contract Type

Resolving issue

Previous credit

Age Range

Select a candidate

Is in client

Contract

clients correspond to use filter



XGBClassifier

Customer ID: 123456
Name: John
Age: 35 years
Type of Contract: Resolving issue
Number of children: 1
Total income: \$20000
probability of default: 0.1
Customer status: Eligible